



ADVANCED DATABASE SYSTEMS

WEEK 5

QUERY PROCESSING

PART IV



Join Operators



Join Operators

- Joins are a very common query operation.
- Joins can be very expensive:

Consider an inner join of R and S each with 1M records.

Q: How many tuples in the answer?

(cross product in worst case, 0 in the best(?))

- Many join algorithms have been developed
- Can have very different join costs.



Equality Joins With One Join Column

```
SELECT *  
FROM   Reserves R1, Sailors S1  
WHERE  R1.sid=S1.sid
```

- In algebra: $R \bowtie S$. Common! Must be carefully optimized. $R \times S$ is large; so, $R \times S$ followed by a selection is inefficient.
- Assume:
 - $M = 1000$ pages in R , $p_R = 100$ tuples per page.
 - $N = 500$ pages in S , $p_S = 80$ tuples per page.
 - In our examples, R is Reserves and S is Sailors.
- *Cost metric*: # of I/Os. We will ignore output costs.
- We will consider more complex join conditions later.



Simple Nested Loops Join

```
foreach tuple r in R do
    foreach tuple s in S do
        if  $r_i == s_j$  then add  $\langle r, s \rangle$  to result
```

- ❑ For each tuple in the *outer* relation R, we scan the entire *inner* relation S.
- ❑ How much does this Cost?
- ❑ $(p_R * M) * N + M = 100,000 * 500 + 1000$ I/Os. (about 50M I/Os!!)
- ❑ What if smaller relation (S) was outer?
- ❑ $(p_S * N) * M + N = 40,000 * 1000 + 500$ I/Os. (better.... 😊 40M I/Os)
- ❑ Prohibitively expensive...

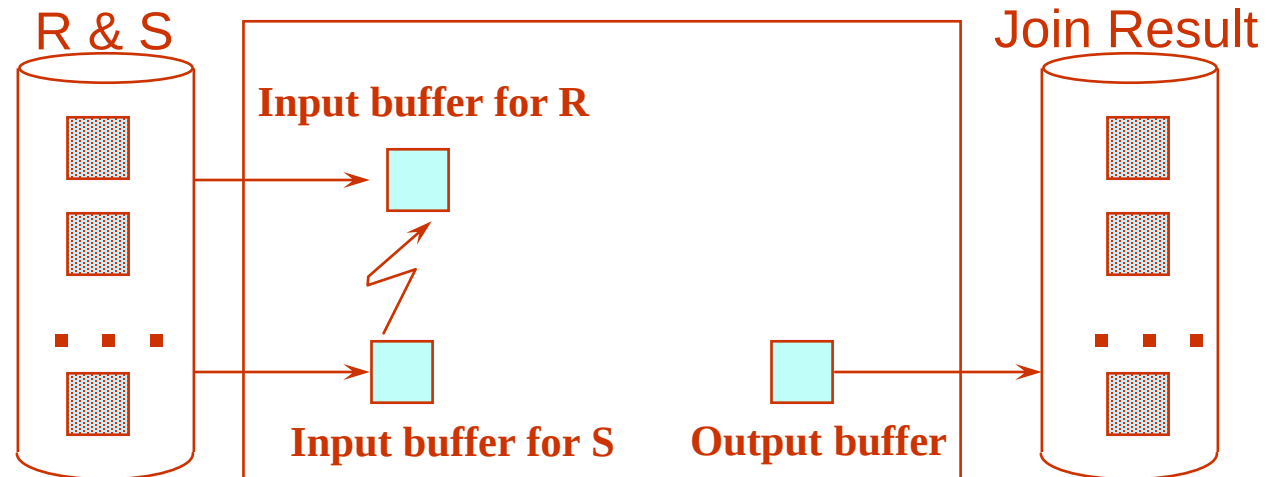
Q: What is the cost if one relation can fit entirely in memory?
 $M+N = 1500$ I/Os!!!!



Page-Oriented Nested Loops Join

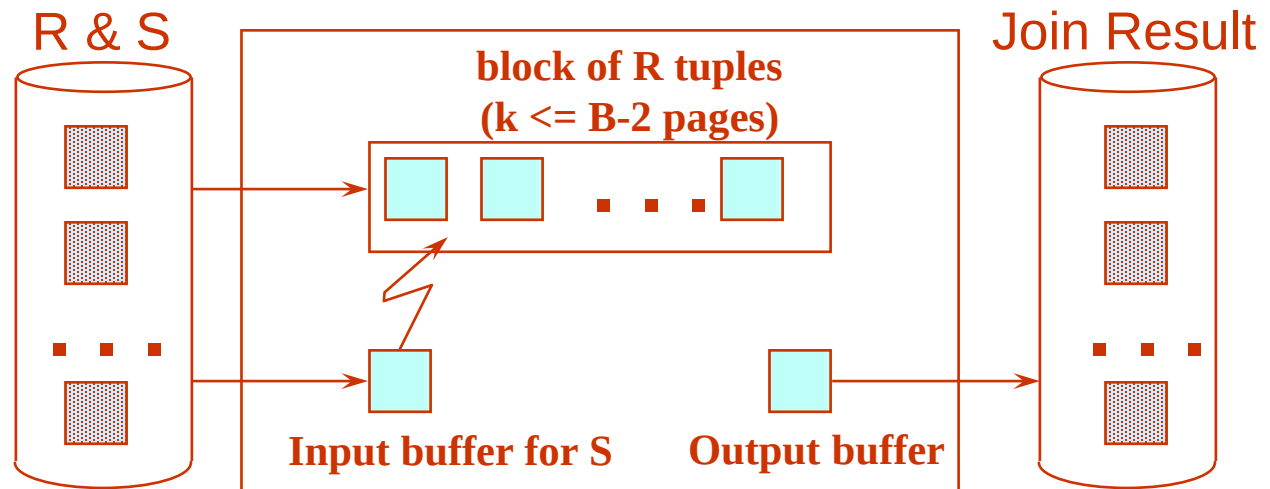
```
foreach page  $b_R$  in R do
  foreach page  $b_S$  in S do
    foreach tuple  $r$  in  $b_R$  do
      foreach tuple  $s$  in  $b_S$  do
        if  $r_i == s_j$  then add  $\langle r, s \rangle$  to result
```

- For each *page* of R, get each *page* of S, and write out matching pairs of tuples $\langle r, s \rangle$, where r is in R-page and S is in S-page.
- What is the cost of this approach?
- $M*N + M = 1000*500 + 1000 = 501000$
 - If smaller relation (S) is outer, cost = $500*1000 + 500 = 500500$



Block Nested Loops Join

- ❑ Page-oriented NL doesn't exploit extra buffers.
- ❑ **Alternative approach:** Use one page as an input buffer for scanning the inner S , one page as the output buffer, and use all remaining pages to hold ``block'' of outer R .
- ❑ For each matching tuple r in R -block, s in S -page, add $\langle r, s \rangle$ to result. Then read next R -block, scan S , etc.



Index Nested Loops Join

```
foreach tuple r in R do
    foreach tuple s in S where  $r_i == s_j$  do
        add <r, s> to result
```

- ❑ If there is an index on the join column of one relation (say S), can make it the inner and exploit the index.
 - ❑ Cost: $M + (M * p_R) * \text{cost of finding matching S tuples}$
- ❑ For each R tuple, cost of probing S index is about 1.2 for hash index, 2-4 for B+ tree.
- ❑ Cost of then finding S tuples depends on clustering.
 - ❑ Clustered index: 1 I/O per page of matching S tuples.
 - ❑ Unclustered: up to 1 I/O per matching S tuple.



Sort-Merge Join ($R \bowtie_{i=j} S$)

- Sort R and S on the join column, then scan them to do a “merge” (on join col.), and output result tuples.
 - Advance scan of R until current R-tuple $\geq$ current S tuple,
 - Then advance scan of S until current S-tuple $\geq$ current R tuple;
 - do this until current R tuple = current S tuple.
 - At this point, all R tuples with same value in R_i (*current R group*) and all S tuples with same value in S_j (*current S group*) match;
 - output $\langle r, s \rangle$ for all pairs of such tuples.
 - Then resume scanning R and S.
- R is scanned once;
- Each S group is scanned once per matching R tuple.
 - (Multiple scans of an S group are likely to find needed pages in buffer.)
- Particularly useful if
 - one or both inputs are already sorted on join attribute(s)
 - output is required to be sorted on join attributes(s)



Example of Sort-Merge Join

<u>sid</u>	sname	rating	age
22	dustin	7	45.0
28	yuppy	9	35.0
31	lubber	8	55.5
44	guppy	5	35.0
58	rusty	10	35.0

<u>sid</u>	<u>bid</u>	<u>day</u>	rname
28	103	12/4/96	guppy
28	103	11/3/96	yuppy
31	101	10/10/96	dustin
31	102	10/12/96	lubber
31	101	10/11/96	lubber
58	103	11/12/96	dustin

- Cost: Sort S + Sort R + (M + N)
 - The cost of merging: usually M + N,
 - ▶ worst case is M*N (but very unlikely!)
- With 35, 100 or 300 buffer pages, both Reserves and Sailors can be sorted in 2 passes; total join cost: 7500.

(BNL cost: 2500 to 16500 I/Os)



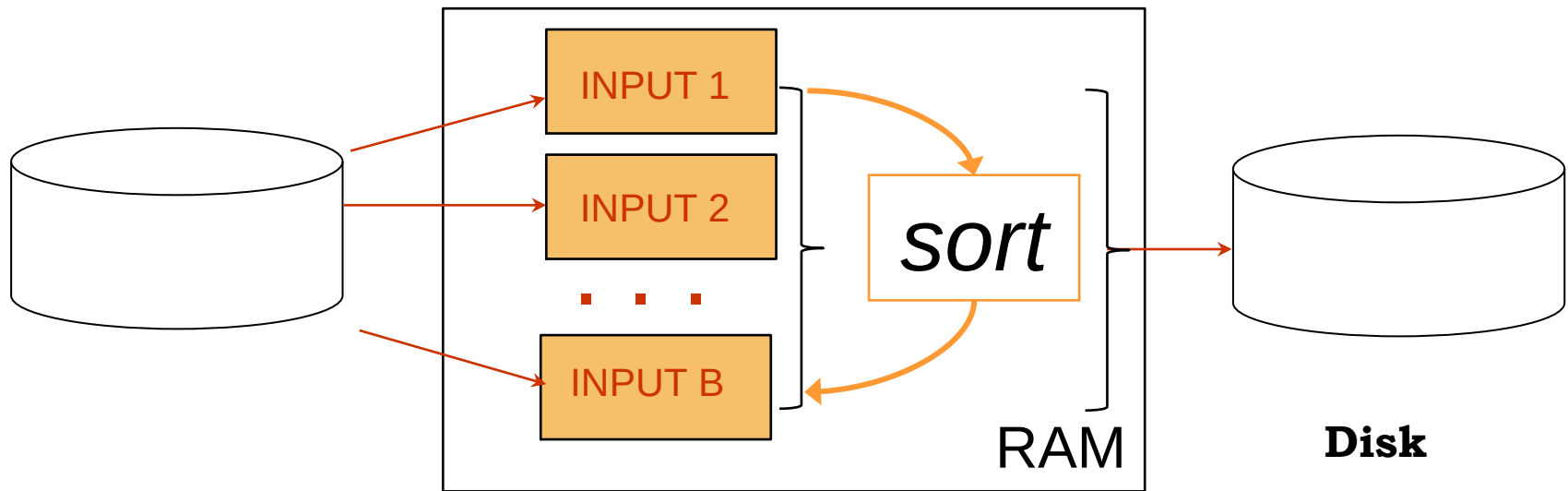
Cost of Sort-Merge

- For $B = 35$
 - Sort-Merge:
 - ▶ sort R: in two passes $\Rightarrow 4M = 4000$
 - ▶ sort S: in two passes $\Rightarrow 4N = 2000$
 - ▶ merge: $M + N$ (hopefully...) $\Rightarrow 1500$
 - ▶ Total: 7500
 - Block Nested Loop Join:
 - ▶ $\text{celing}(N/B-2)*M+N = 16*1000+500 = 16500$
 - Sort-Merge Better for $B=35!!!!$
- For $B = 300$
 - Sort-Merge: the same: 7500
 - BNLJ:
 - ▶ $\text{celing}(N/B-2)*M+N = 2*1000+500 = 2500$
 - Here BNLJ is better!!!!



Resresher: General External Merge Sort

- More than 3 buffer pages. How can we utilize them?
- To sort a file with N pages using B buffer pages:
 - Pass 0: use B buffer pages. Produce $\lceil N / B \rceil$ sorted runs of B pages each.



Pass 0 – Create Sorted Runs

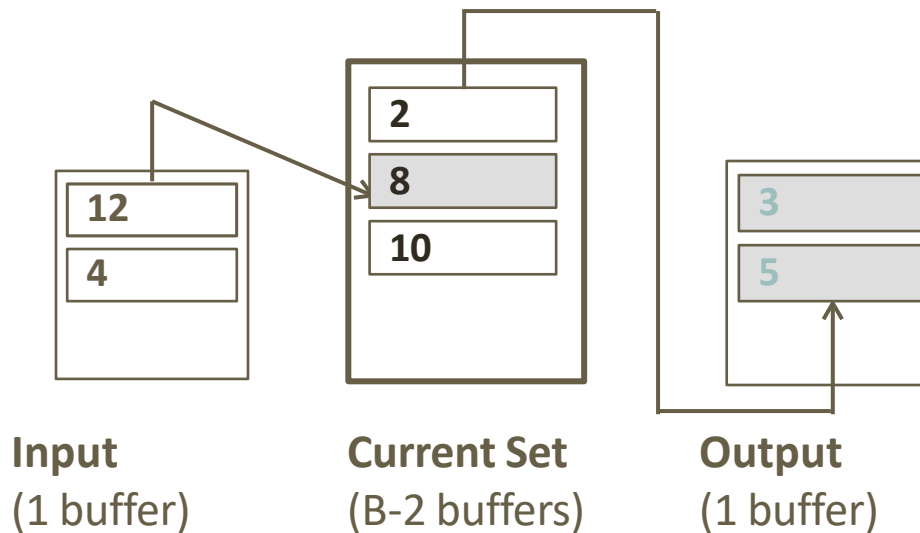
Refresher: Internal Sort Algorithm

- ❑ Quicksort is a fast way to sort in memory.
- ❑ An alternative is “tournament sort” (a.k.a. “heapsort”)
 - ❑ **Top:** Read in ***B*** blocks and build a min-heap
 - ❑ **Output:** move smallest record to output buffer
 - ❑ Read in a new record *r*
 - ❑ insert *r* into “heap”
 - ❑ if *r* is not the smallest, then **GOTO Output**
 - ❑ else remove *r* from “heap”
 - ❑ output “heap” in order; **GOTO Top**



Replacement Sort

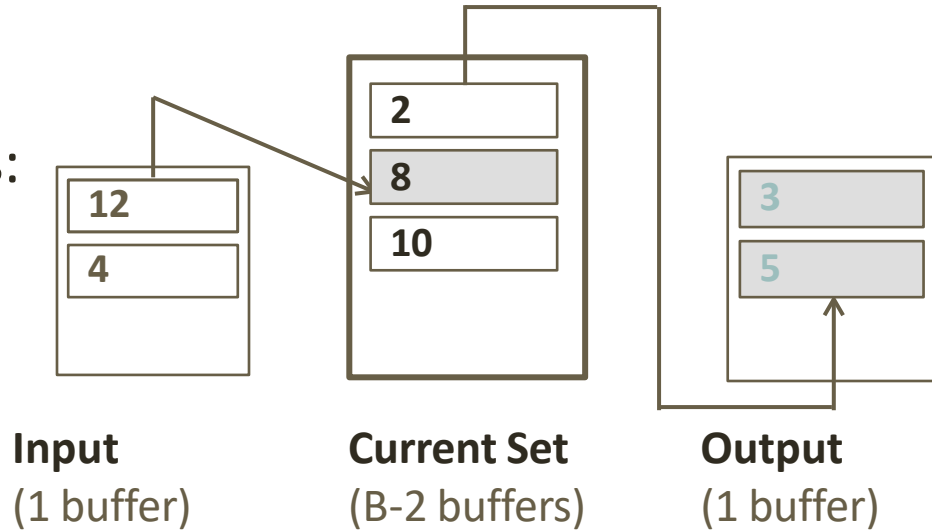
- ❖ Produces initial sorted runs as long as possible.
- ❖ Replacement Sort: when used in Pass 0 for sorting, can write out sorted runs of size $2B$ on average.
 - Affects calculation of the number of passes accordingly.



Replacement Sort

- Organize B available buffers:

- 1 buffer for *input*
- B-2 buffers for *current set*
- 1 buffer for *output*



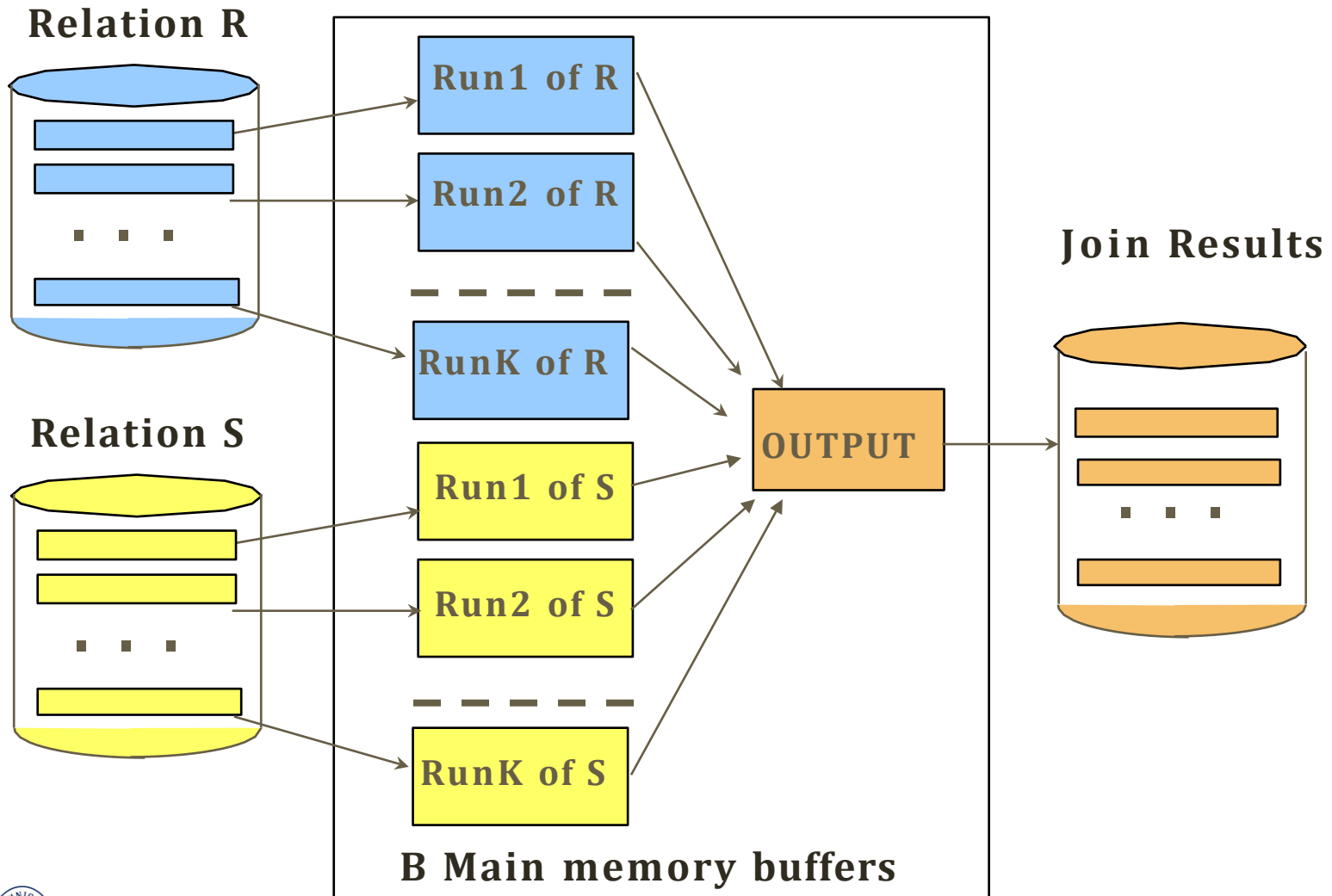
- ❖ Pick tuple r in the current set with the *smallest value that is $\geq$ largest value in output*, e.g. 8, to extend the current run.
- ❖ Fill the space in current set by adding tuples from input.
- ❖ Write output buffer out if full, extending the current run.
- ❖ Current run terminates if *every tuple in the current set is smaller than the largest tuple in output*.

Refinement of Sort-Merge Join

- We can combine the merging phases in the *sorting* of R and S with the merging required for the join.
 - Pass 0 as before, but apply to both R then S before merge.
 - ▶ If $B \geq \sqrt{L}$, where L is the size of the larger relation, using the sorting refinement that produces runs of length $2B$ in Pass 0, #runs of each relation is $< B/2$.
 - In “Merge” phase: Allocate 1 page per run of **each relation**, and ‘merge’ while checking the join condition
 - **Cost**: read+write each relation in Pass 0 + read each relation in (only) merging pass (+ writing of result tuples).
 - In example, cost goes down from 7500 to 4500 I/Os for $B=300$.
- In practice, the I/O cost of sort-merge join, like the cost of external sorting, is *linear*.



Refinement of Sort-Merge Join



Hash-Join

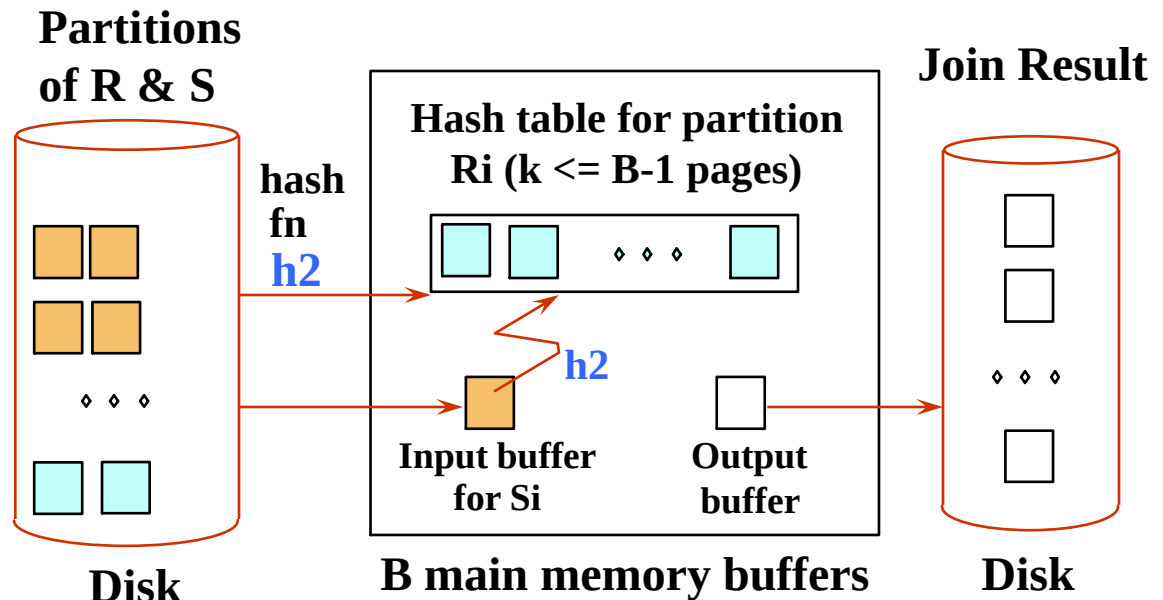
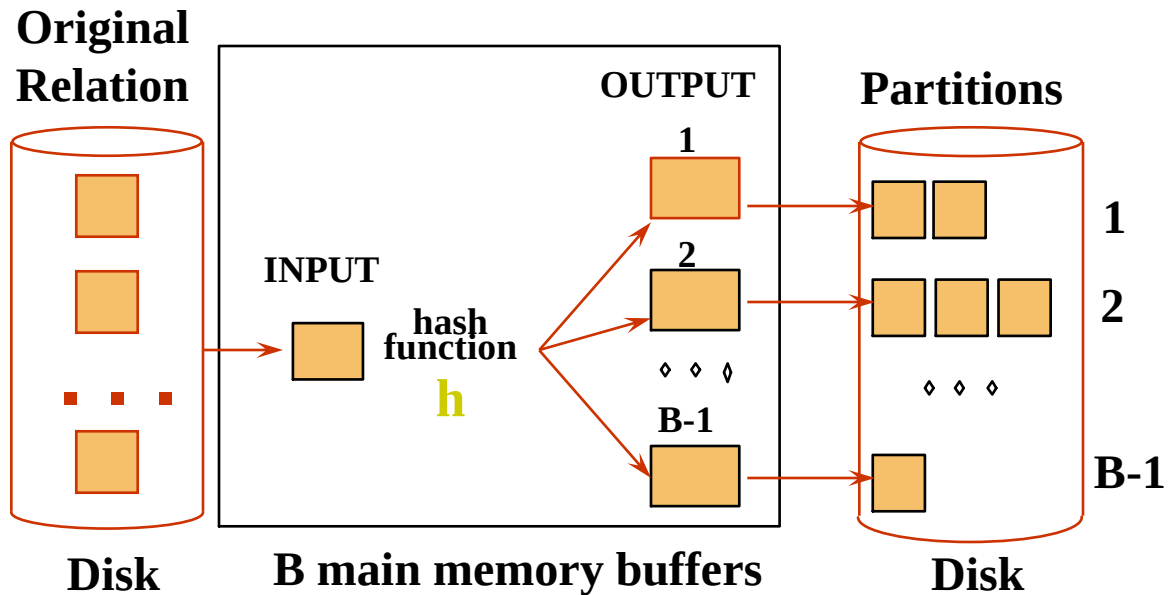
- Partition both relations on the join attributes using hash function h .
- R tuples in partition R_i will **only** match S tuples in partition S_i .

For $i = 1$ to #partitions {

Read in partition R_i
and hash it using h_2 (not h).

Scan partition S_i and
probe hash table for matches.

}



Observations on Hash-Join

- #partitions $k < B$, and $B-2 > \text{size of smaller partition}$ to be held in memory.
Assuming uniformly sized partitions, and maximizing k , we get:
 $k = B-1$, and $M/(B-1) < B-2$, i.e., B must be $> \sqrt{M}$
- Since we build an in-memory hash table to speed up the matching of tuples in the second phase, a little more memory is needed.



Handling of Overflows

- Partitioning is said to be **skewed** if some partitions have significantly more tuples than some others
- **Hash-table overflow** occurs in partition s_i if s_i does not fit in memory. Reasons could be
 - Many tuples in s with same value for join attributes
 - Bad hash function
- **Overflow resolution** can be done in build phase
 - Partition s_i is further partitioned using different hash function.
 - Partition r_i must be similarly partitioned.
- **Overflow avoidance** performs partitioning carefully to avoid overflows during build phase
 - e.g. partition build relation into many partitions, then combine them
- Both approaches fail with large numbers of duplicates
 - Fallback option: use block nested loops join on overflowed partitions



Cost of Hash-Join

- In partitioning phase, read+write both relns; $2(M+N)$. In matching phase, read both relns; $M+N$ I/Os.
- In our running example, this is a total of 4500 I/Os.
- Sort-Merge Join vs. Hash Join:
 - Given a minimum amount of memory, both have a cost of $3(M+N)$ I/Os. Hash Join superior if relation sizes differ greatly (e.g., if one reln fits in memory). Also, Hash Join shown to be highly parallelizable.
 - Sort-Merge less sensitive to data skew; result is sorted.



Hybrid Hash Join

- Same as partitioned hash join except:
 - Joining phase of one of the partitions is included during the partitioning phase.
- **Partitioning phase:**
 - Allocate buffers for smaller relation- one block for each of the $M-1$ partitions, remaining blocks to partition 1.
 - Repeat for the larger relation in the pass through S .
- **Joining phase:**
 - $M-1$ iterations are needed for the partitions $R_2, R_3, R_4, \dots, R_m$ and $S_2, S_3, S_4, \dots, S_m$.
 - R_1 and S_1 are joined during the partitioning of S_1 , and results of joining R_1 and S_1 are already written to the disk by the end of partitioning phase.



General Join Conditions

- Equalities over several attributes
- (e.g., *R.sid=S.sid AND R.rname=S.sname*):
 - For Index NL, build index on *<sid, sname>* (if S is inner); or use existing indexes on *sid* or *sname*.
 - For Sort-Merge and Hash Join, sort/partition on combination of the two join columns.
- Inequality conditions (e.g., *R.rname < S.sname*):
 - For Index NL, need (clustered!) B+ tree index.
 - Range probes on inner; # matches likely to be much higher than for equality joins.
 - Hash Join, Sort Merge Join not applicable!
 - Block NL quite likely to be the best join method here.



Complex Joins

- Join with a conjunctive condition:

$$r \bowtie_{\theta_1 \wedge \theta_2 \wedge \dots \wedge \theta_n} s$$

- Either use nested loops/block nested loops, or
- Compute the result of one of the simpler joins $r \bowtie_{\theta_i} s$
 - final result comprises those tuples in the intermediate result that satisfy the remaining conditions

$$\theta_1 \wedge \dots \wedge \theta_{i-1} \wedge \theta_{i+1} \wedge \dots \wedge \theta_n$$

- Join with a disjunctive condition

$$r \bowtie_{\theta_1 \vee \theta_2 \vee \dots \vee \theta_n} s$$

- Either use nested loops/block nested loops, or
- Compute as the union of the records in individual joins $r \bowtie_{\theta_i} s$:
 $(r \bowtie_{\theta_1} s) \cup (r \bowtie_{\theta_2} s) \cup \dots \cup (r \bowtie_{\theta_n} s)$



Outer Joins

- ❑ **Outer Join Operators:**
 - ❑ **LEFT OUTER JOIN**
 - ❑ **RIGHT OUTER JOIN**
 - ❑ **FULL OUTER JOIN.**
- ❑ The full outer join produces a result which is equivalent to the union of the results of the left and right outer joins.

- ❑ Example:

```
SELECT      FNAME, DNAME
FROM        (EMPLOYEE LEFT OUTER JOIN DEPARTMENT
               ON DNO = DNUMBER);
```

- ❑ Note: The result of this query is a table of employee names and their associated departments. It is similar to a regular join result, with the exception that if an employee does not have an associated department, the employee's name will still appear in the resulting table, although the department name would be indicated as null.



Outer Joins

- Executing a combination of relational algebra operators.
- Implement the previous left outer join example
 - {Compute the JOIN of the EMPLOYEE and DEPARTMENT tables}
 - ▶ $TEMP1 \leftarrow \pi_{FNAME, DNAME} (EMPLOYEE \bowtie_{DNO=DNUMBER} DEPARTMENT)$
 - {Find the EMPLOYEEs that do not appear in the JOIN}
 - ▶ $TEMP2 \leftarrow \pi_{FNAME} (EMPLOYEE) - \pi_{FNAME} (Temp1)$
 - {Pad each tuple in TEMP2 with a null DNAME field}
 - ▶ $TEMP2 \leftarrow TEMP2 \times 'null'$
 - {UNION the temporary tables to produce the LEFT OUTER JOIN}
 - ▶ $RESULT \leftarrow TEMP1 \cup TEMP2$
- The cost of the outer join, as computed above, would include the cost of the associated steps (i.e., join, projections and union).



Outer Joins

□ Modifying Join Algorithms:

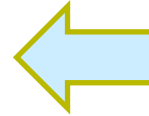
- Nested Loop or Sort-Merge joins can be modified to implement outer join. E.g.,
 - ▶ For left outer join, use the left relation as outer relation and construct result from every tuple in the left relation.
 - ▶ If there is a match, the concatenated tuple is saved in the result.
 - ▶ However, if an outer tuple does not match, then the tuple is still included in the result but is padded with a null value(s).



Set Operations

- Intersection and cross-product special cases of join.

- SELECT R.A, R.B
FROM R, S
WHERE R.A = S.A and R.B = S.B



```
SELECT R.A, R.B  
FROM R  
INTERSECT  
SELECT S.A, S.B  
FROM S
```

- Union (Distinct) and Except similar; we'll do union.
- Sorting based approach to union:
 - Sort both relations (on combination of all attributes).
 - Scan sorted relations and merge them.
 - Alternative:* Merge runs from Pass 0 for *both* relations.
- Hash based approach to union:
 - Partition R and S using hash function h .
 - For each S-partition, build in-memory hash table (using h_2), scan corr. R-partition and add tuples to table while discarding duplicates.



Aggregation Operators



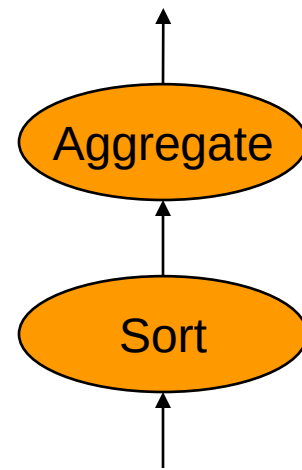
Aggregate Operations (AVG, MIN, etc.)

- Without grouping:
 - In general, requires scanning the relation (FULL SCAN or HEAP SCAN).
 - Given a tree index whose search key includes all attributes in the SELECT and WHERE clauses, can do index-only scan.
- With grouping:
 - Sort on group-by attributes, then scan relation and compute aggregate for each group.
 - Similar approach based on hashing on group-by attributes.
 - Given a tree index whose search key includes all attributes in SELECT, WHERE and GROUP BY clauses, can do index-only scan; if group-by attributes form prefix of search key, can retrieve data entries/tuples in group-by order.



Sort-based GROUP BY

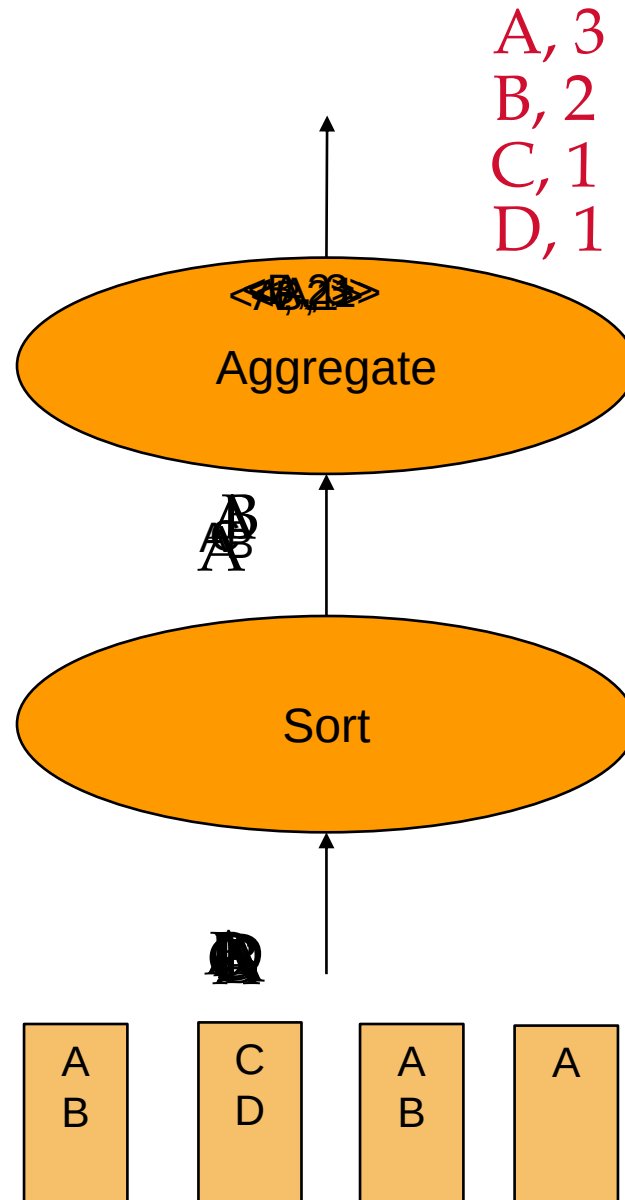
Naïve Solution



- The Sort iterator naturally permutes its input so that all tuples are output in sequence
- The Aggregate iterator keeps running info (“**transition values**” or “**transVals**”) on agg functions in the SELECT list, per group: Example transVals:
 - For COUNT, it keeps count-so-far
 - For SUM, it keeps sum-so-far
 - For AVERAGE it keeps sum-so-far *and* count-so-far
- As soon as the Aggregate iterator sees a tuple from a new group:
 1. It produces an output for the old group based on the agg function
 - e.g. for AVERAGE it returns (sum-so-far/count-so-far)
 2. It resets its running info.
 3. It updates the running info with the new tuple's info



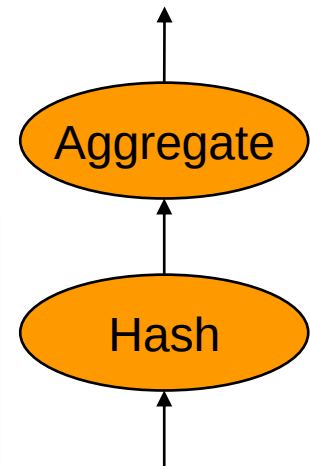
Sort-based GROUP BY: Naïve Solution



Hash-based GROUP BY

Naïve Solution

- Similar to the Sort-based GROUPBY
 - The Hash iterator → all tuples are output in groups.
- The Aggregate iterator keeps running info (“**transition values**” or “**transVals**”) on aggregation functions in the SELECT list per group:
- Example transVals:
 - For COUNT, it keeps count-so-far
 - For SUM, it keeps sum-so-far
 - For AVERAGE it keeps sum-so-far **and** count-so-far
- As soon as the Aggregate iterator sees a tuple from a new group:
 1. Produce an output for the old group based on the agg function
e.g. for AVERAGE it returns (sum-so-far/count-so-far)
 2. Reset running info
 3. Update the running info with the new tuple's info

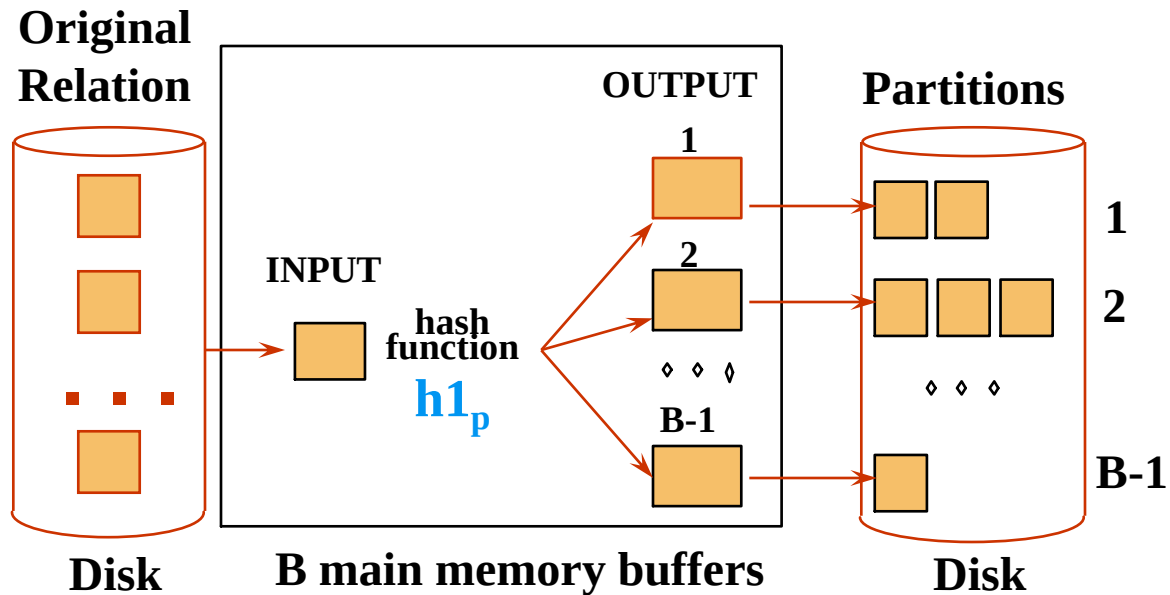


The same as
the sort-based
group by

External Hashing

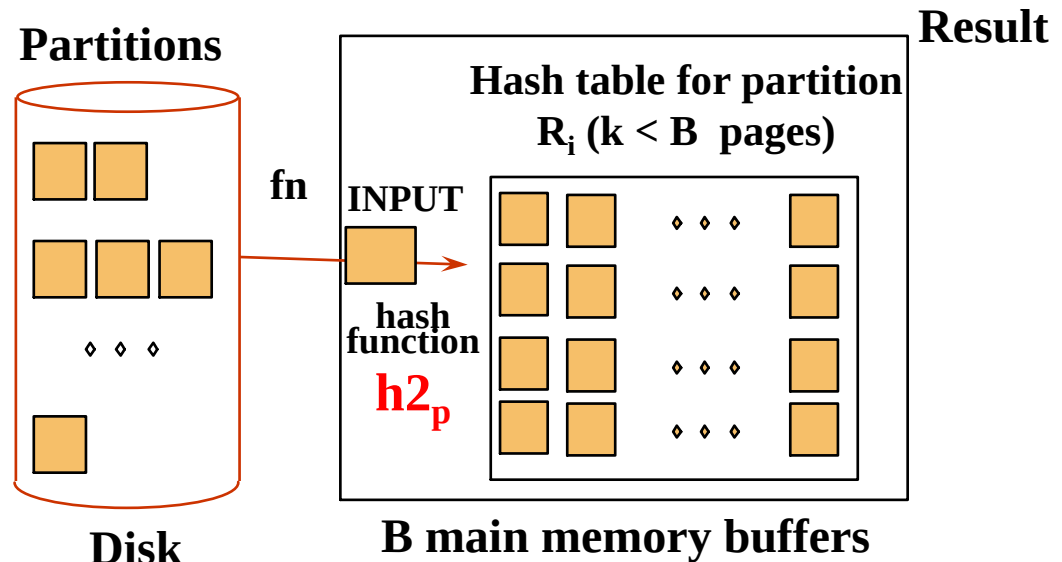
Partition:

- Each group will be in a single disk-based partition file. But those files have many groups inter-mixed.

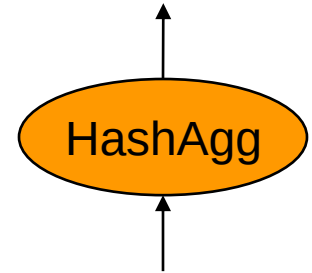


Rehash:

- For Each Partition i :
 - hash i into an in-memory hash table
 - Return results until records exhausted then $i++$



We Can Do Better!



- Put summarization into the hashing process
 - During the ReHash phase, don't store tuples, store pairs of the form **<GroupVals, TransVals>**
 - When we want to insert a new tuple into the hash table
 - ▶ If we find a matching GroupVals, just update the TransVals appropriately
 - ▶ Else insert a new <GroupVals,TransVals> pair
- What's the benefit?
 - Q: How many pairs will we have to maintain in the rehash phase?
 - A: Number of **distinct values** of GroupVals columns
 - ▶ Not the number of tuples!!
 - Also probably “narrower” than the tuples



Projection (DupElim)

```
SELECT DISTINCT R.sid, R.bid  
FROM Reserves R
```

- Issue is removing **duplicates**.
- Basic approach is to use sorting
 1. Scan R, extract only the needed attrs (why do this 1st?)
 2. Sort the resulting set
 3. Remove adjacent duplicates
- Cost: Reserves with size ratio 0.25 = 250 pages. With 20 buffer pages, can sort in 2 passes, so
$$1000 + 250 + 2 * 2 * 250 + 250 = 2500 \text{ I/Os}$$
- Can improve by modifying external sort algorithm:
 - Modify Pass 0 of external sort to eliminate unwanted fields.
 - Modify merging passes to eliminate duplicates.
 - Cost: for above case: read 1000 pages, write out 250 in runs of 20 pages, merge runs = $1000 + 250 + 250 = 1500$.



DupElim & Indexes

- Sort-based approach is the standard; better handling of skew and result is sorted.
- If an index on the relation contains all wanted attributes in its search key, can do *index-only* scan.
 - Apply projection techniques to data entries (much smaller!)
- If an ordered (i.e., tree) index contains all wanted attributes as *prefix* of search key, can do even better:
 - Retrieve data entries in order (index-only scan), discard unwanted fields, compare adjacent tuples to check for duplicates.
- Same tricks apply to GROUP BY/Aggregation



Review

- Implementation of Relational Operations
 - Focus largely on External algorithms (sorting/hashing)
- Joins
 - Blocked nested loops:
 - ▶ simple, exploits extra memory
 - Indexed nested loops:
 - ▶ best if one relation small and one indexed
 - Sort/Merge Join
 - ▶ good with small amount of memory, bad with duplicates
 - Hash Join
 - ▶ fast (if enough memory), bad with skewed data
 - ▶ Relatively easy to parallelize



Review

- *Queries are composed of a few basic operators;*
 - The implementation of these operators can be carefully tuned (and it is important to do this!).
 - Operators are “plug-and-play” due to the *Iterator* model.
- Many alternative implementation techniques for each operator; no universally superior technique for most.
- Must consider alternatives for each operation in a query and choose best one.
- Choices depend on indexes, memory, stats,...
- This is part of the broader task of Query Optimization, which we will cover next!

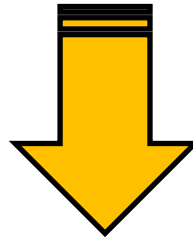


QUERY OPTIMIZATION PART I



Query Optimization

For a given SQL Query



Goal:

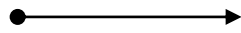
Build the most “optimal” execution plan for the query



Query Optimization

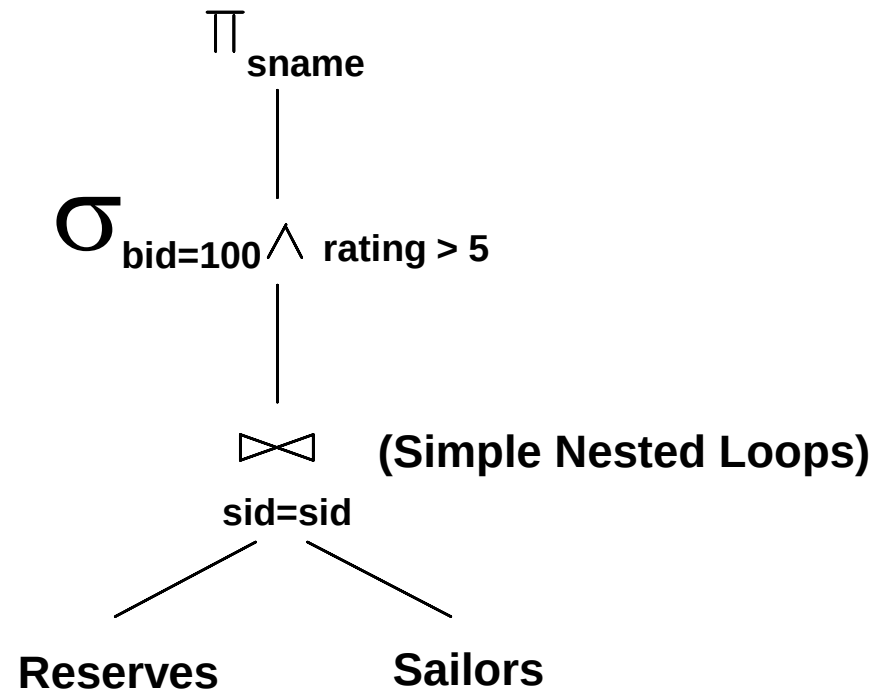
Goal:

Declarative SQL query



Imperative query execution plan:

```
SELECT S.sname
FROM Reserves R, Sailors S
WHERE R.sid = S.sid
      AND R.bid = 100
      AND S.rating > 5
```

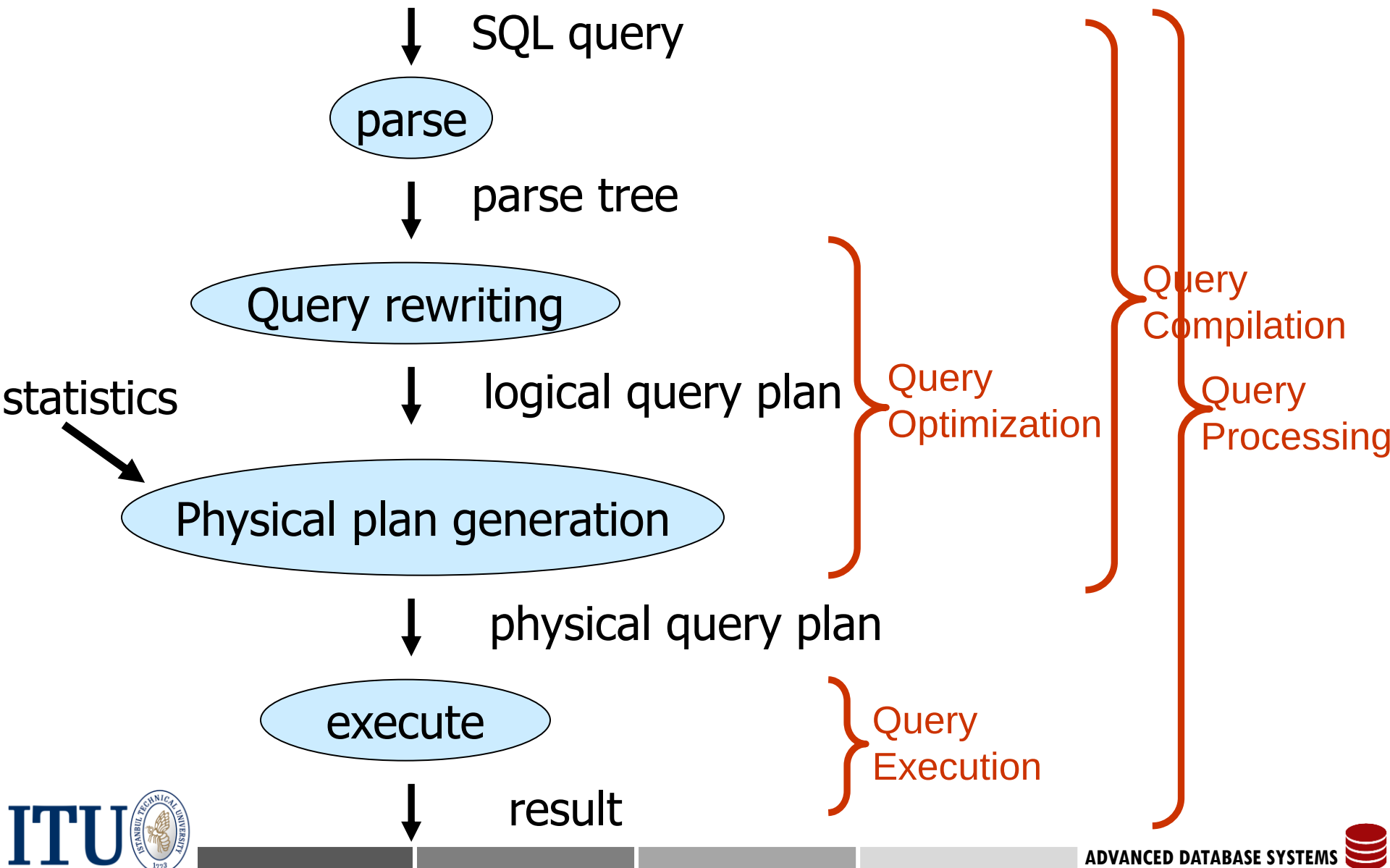


Plan: Tree of R.A. ops, with choice of alg for each op.

Ideally: Want to find best plan. **Practically:** Avoid worst plans!



Overview of Query Processing



How do we execute this query?

Select B,D

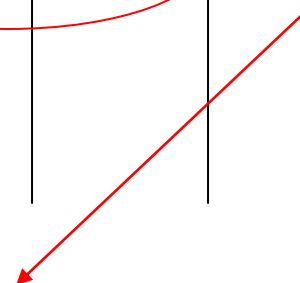
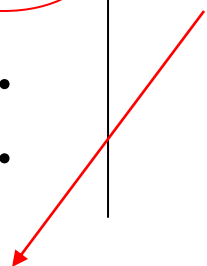
From R,S

Where $R.A = \text{"c"} \wedge S.E = 2 \wedge R.C = S.C$

One idea

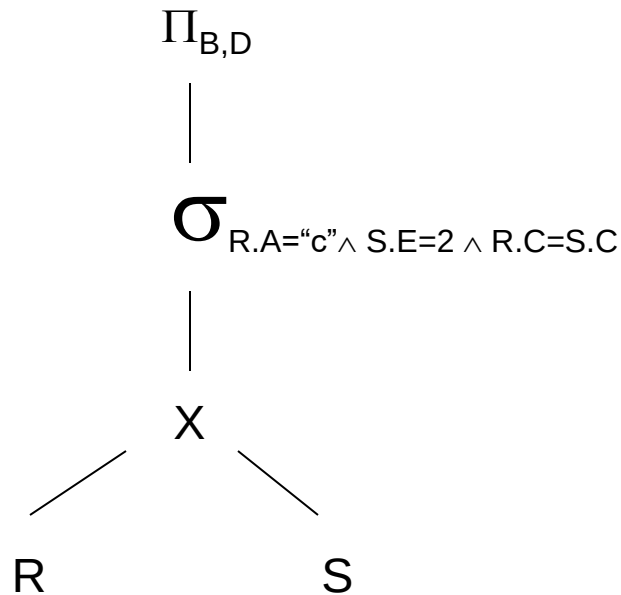
- Do Cartesian product
- Select tuples
- Do projection

R X S	R.A	R.B	R.C	S.C	S.D	S.E
Select B,D	a	1	10	10	x	2
From R,S	a	1	10	20	x	2
Where R.A = "c" $\wedge$	.					
S.E = 2 $\wedge$ R.C=S.C	.					
Bingo! →	c	2	10	10	x	2
Got one...	.					
	.					



Relational Algebra - can be used to describe plans

Ex: Plan I



OR: $\Pi_{B,D} [\sigma_{R.A="c" \wedge S.E=2 \wedge R.C=S.C} (RXS)]$



Another idea:

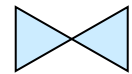
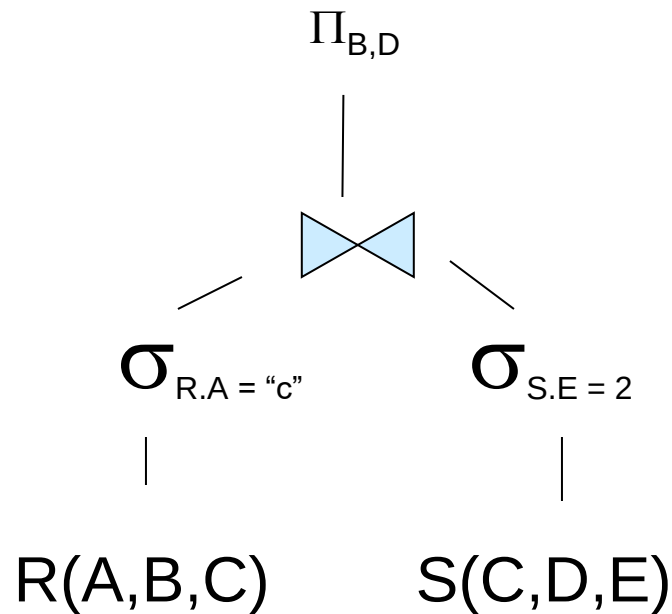
Plan II

Select B,D

From R,S

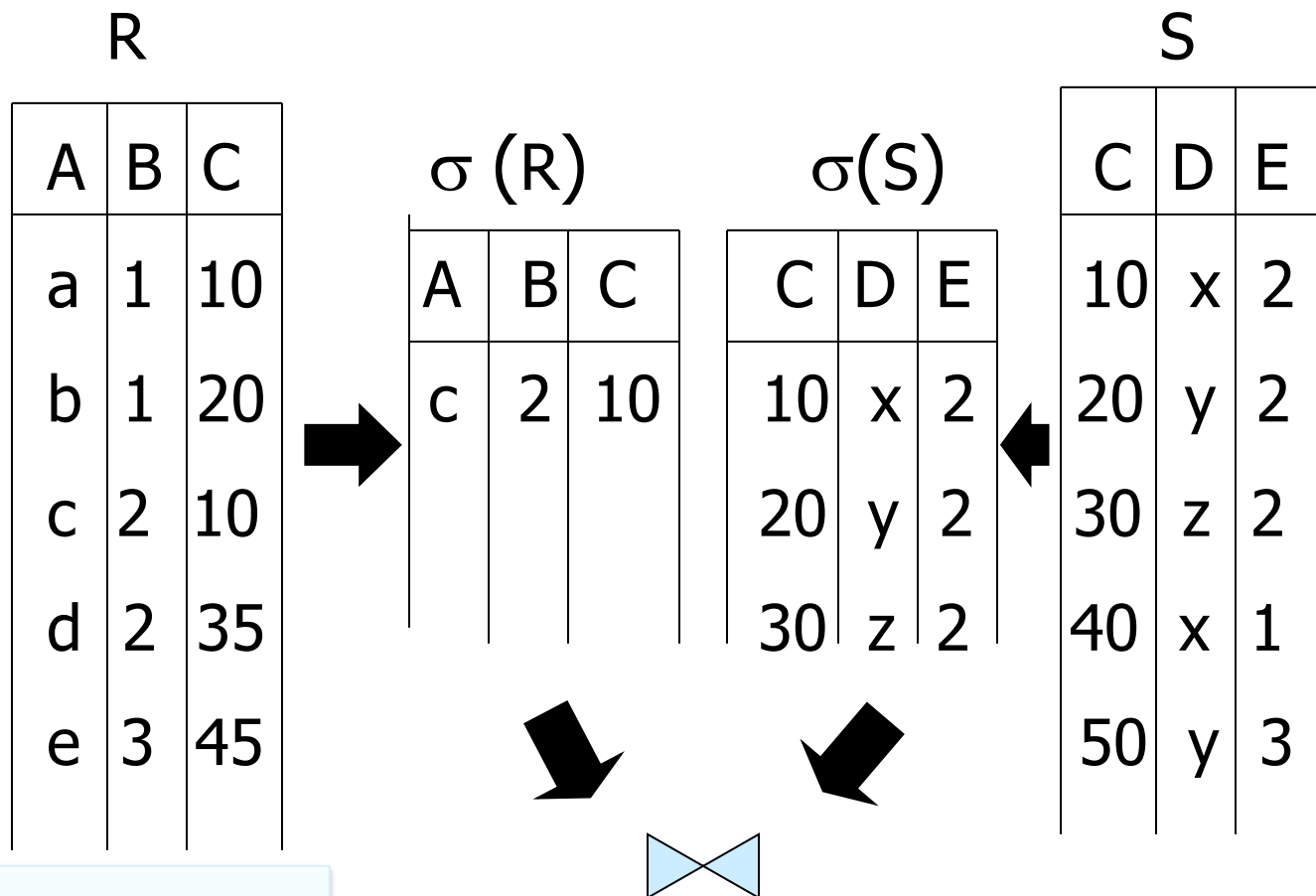
Where $R.A = \text{"c"} \wedge$

$S.E = 2 \wedge R.C = S.C$



natural join





Select B,D

From R,S

Where R.A = "c" $\wedge$

S.E = 2 $\wedge$ R.C=S.C

Plan III

Use R.A and S.C **Indexes**

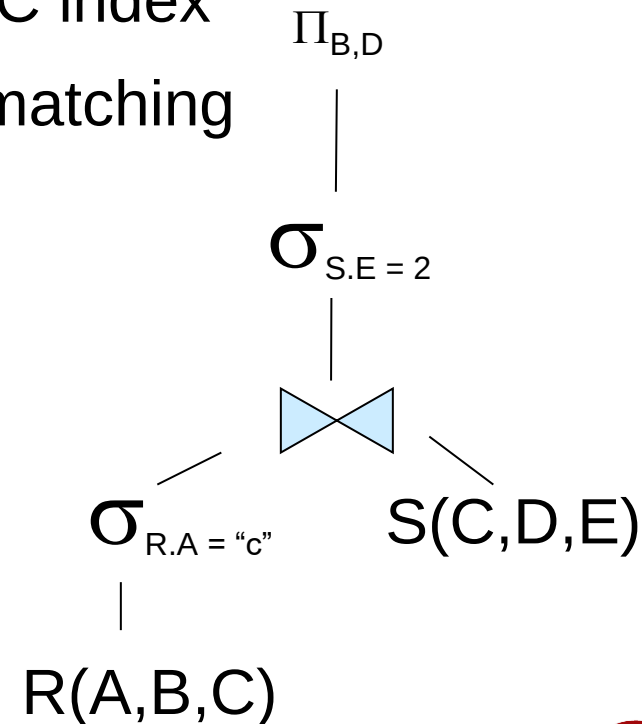
Select B,D
From R,S
Where R.A = "c" $\wedge$
S.E = 2 $\wedge$ R.C=S.C

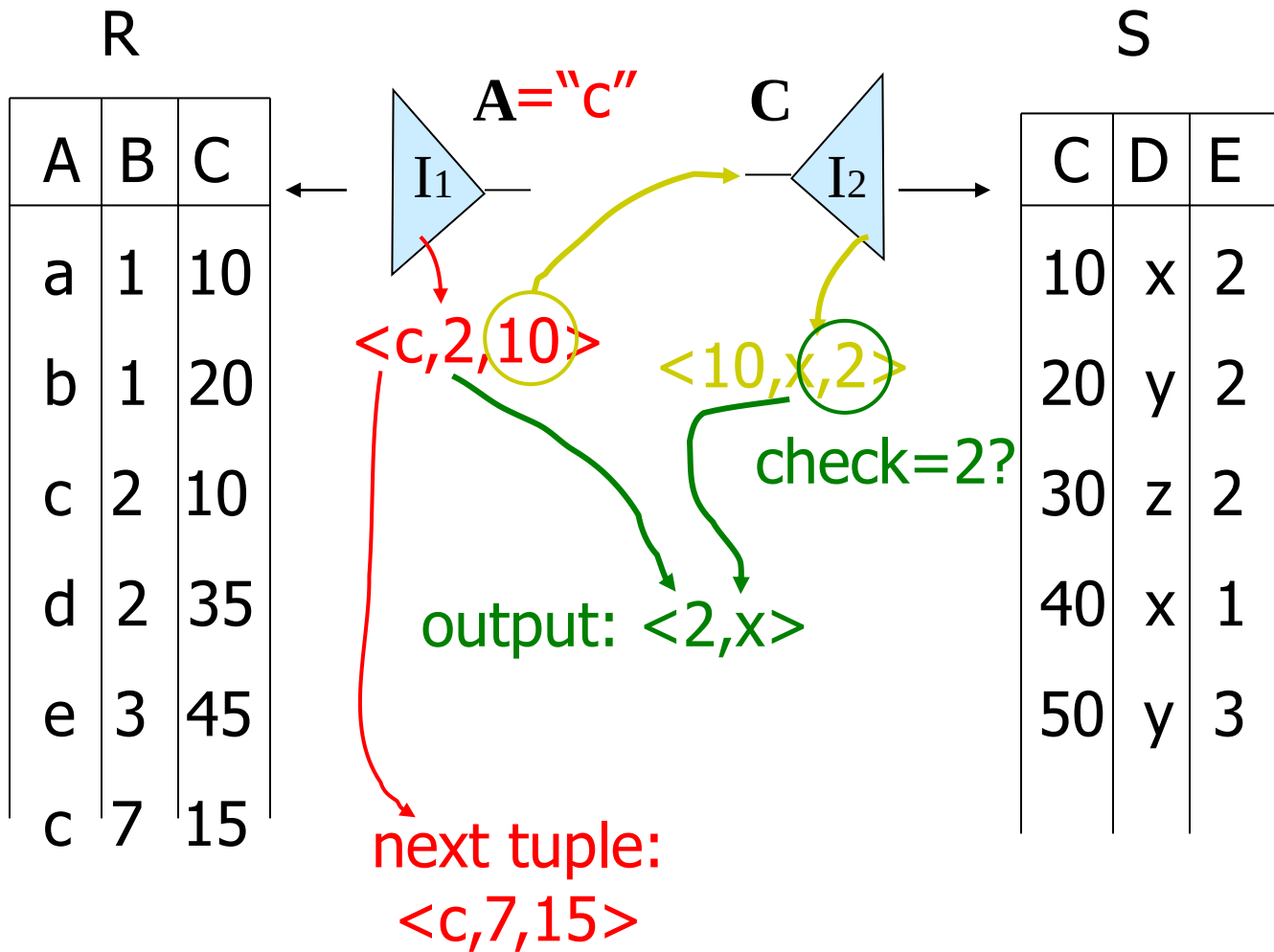
(1) Use R.A index to select R tuples with R.A = "c"

(2) For each R.C value found, use S.C index to find matching tuples, and join matching R, S tuples,

(3) Eliminate S tuples S.E $\neq$ 2

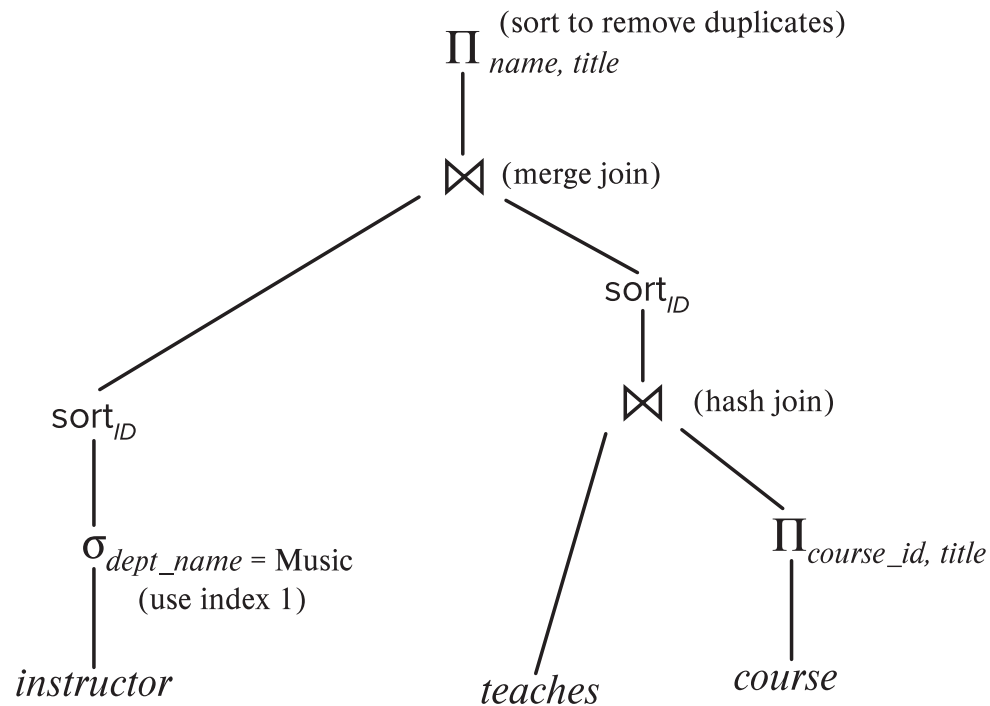
(4) Project B, D attributes,
and place in result





Introduction (Cont.)

- An **evaluation plan** defines exactly what algorithm is used for each operation, and how the execution of the operations is coordinated.

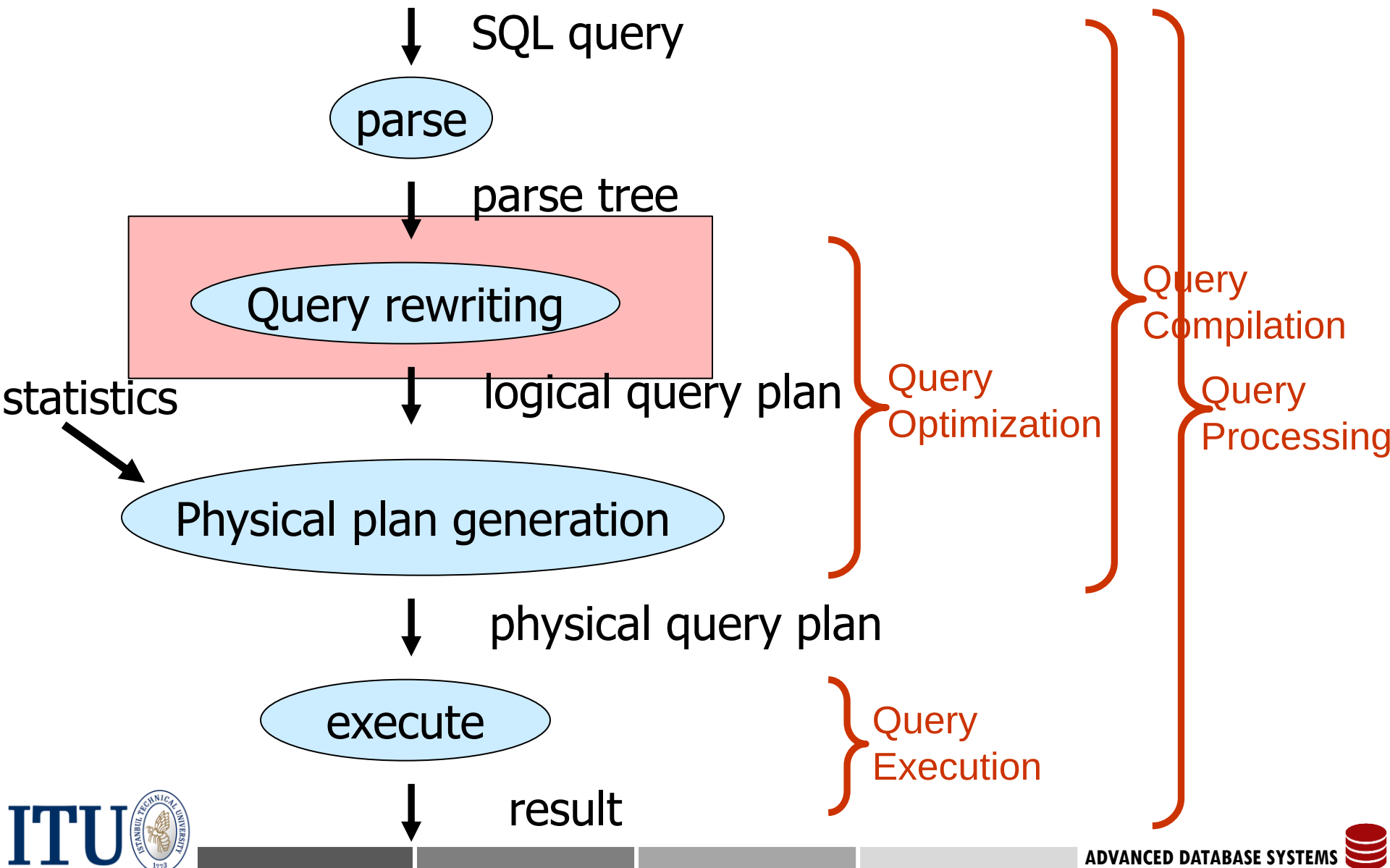


Introduction (Cont.)

- Cost difference between evaluation plans for a query can be enormous
 - e.g., seconds vs. days in some cases
- Steps in **cost-based query optimization**
 1. Generate logically equivalent expressions using **equivalence rules**
 - ▶ Query rewriting:
 - transformations from one SQL query to another one using semantic properties.
 2. Annotate resultant expressions to get alternative query plans
Selecting query execution plan:
 - ▶ done on single query blocks (i.e., S-P-J blocks)
 - ▶ main step: join enumeration
 3. Choose the cheapest plan based on **estimated cost**
 - ▶ Statistical information about relations. Examples:
 - number of tuples, number of distinct values for an attribute
 - ▶ Statistics estimation for intermediate results
 - to compute cost of complex expressions
 - ▶ Cost formula for algorithms, computed using statistics



Overview of Query Processing



Viewing Query Evaluation Plans

- ❑ Most database support **explain** <query>
 - ❑ Displays plan chosen by query optimizer, along with cost estimates
 - ❑ Some syntax variations between databases
 - ▶ Oracle: **explain plan for** <query> followed by **select * from** table (*dbms_xplan.display*)
 - ▶ SQL Server: **set showplan_text on**
- ❑ Some databases (e.g. PostgreSQL) support **explain analyze** <query>
 - ❑ Shows actual runtime statistics found by running the query, in addition to showing the plan
- ❑ Some databases (e.g. PostgreSQL) show cost as *f..l*
 - ❑ *f* is the cost of delivering first tuple and *l* is cost of delivering all results



Viewing Query Evaluation Plans

ORACLE

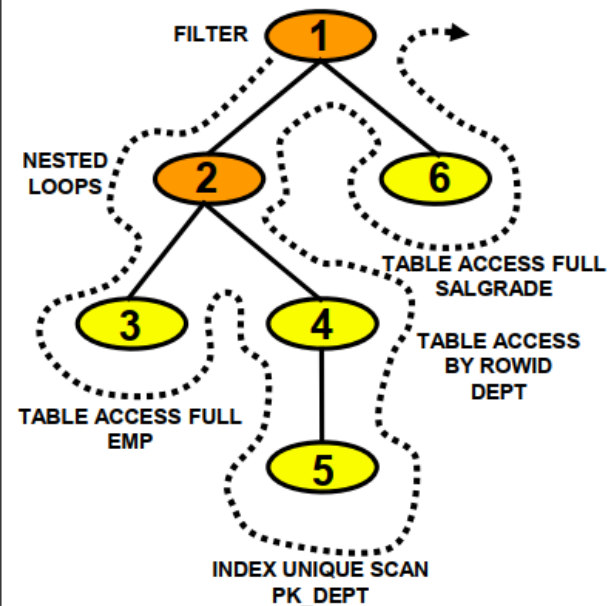
Execution Plan Interpretation

```
SELECT /*+ RULE */ ename,job,sal,dname
FROM emp,dept
WHERE dept.deptno=emp.deptno and not exists(SELECT *
                                           FROM salgrade
                                           WHERE emp.sal between losal and hisal);
```

Id	Operation	Name
0	SELECT STATEMENT	
* 1	FILTER	
2	NESTED LOOPS	
3	TABLE ACCESS FULL	EMP
4	TABLE ACCESS BY INDEX ROWID	DEPT
* 5	INDEX UNIQUE SCAN	PK_DEPT
* 6	TABLE ACCESS FULL	SALGRADE

Predicate Information (identified by operation id):

```
1 - filter( NOT EXISTS
  (SELECT 0 FROM "SALGRADE" "SALGRADE" WHERE
    "HISAL">=:B1 AND "LOSAL"<=:B2))
5 - access("DEPT"."DEPTNO"="EMP"."DEPTNO")
6 - filter("HISAL">=:B1 AND "LOSAL"<=:B2)
```



ORACLE

Source: <https://ittutorial.org/read-or-interpret-an-execution-plan-in-oracle-oracle-database-performance-tuning-tutorial-5>



Generating Equivalent Expressions



Transformation of Relational Expressions

- Two relational algebra expressions are said to be **equivalent** if
 - they generate the same set of tuples on every *legal* database instance
 - **Note:** order of tuples is irrelevant
- In SQL, inputs and outputs are multisets of tuples
 - Two expressions in the multiset version of the relational algebra are said to be equivalent if
 - ▶ the two expressions generate the same multiset of tuples on every legal database instance.
- An **equivalence rule** says that expressions of two forms are equivalent
 - Can replace expression of first form by second, or vice versa



Equivalence Rules

1. Conjunctive selection operations can be deconstructed into a sequence of individual selections.

$$\sigma_{\theta_1 \wedge \theta_2}(E) \equiv \sigma_{\theta_1}(\sigma_{\theta_2}(E))$$

2. Selection operations are commutative.

$$\sigma_{\theta_1}(\sigma_{\theta_2}(E)) \equiv \sigma_{\theta_2}(\sigma_{\theta_1}(E))$$

3. Only the last in a sequence of projection operations is needed, the others can be omitted.

$$\Pi_{L_1}(\Pi_{L_2}(\dots(\Pi_{L_n}(E))\dots)) \equiv \Pi_{L_1}(E)$$

where $L_1 \subseteq L_2 \dots \subseteq L_n$

4. Selections can be combined with Cartesian products and theta joins.

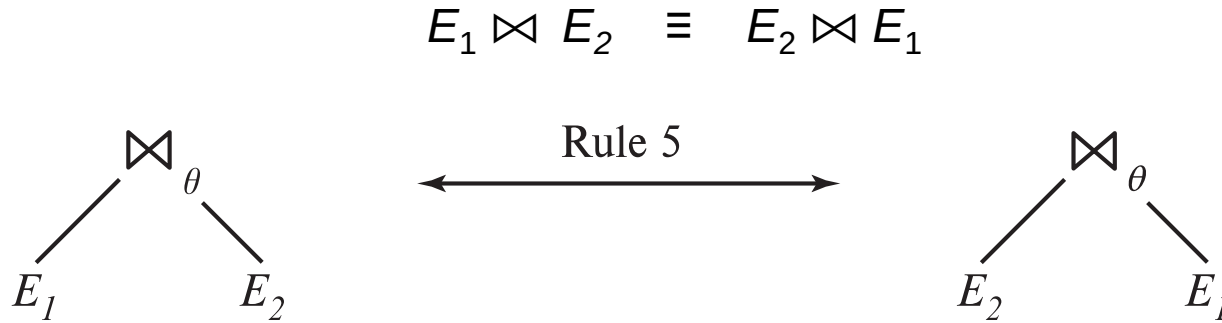
a. $\sigma_{\theta}(E_1 \times E_2) \equiv E_1 \bowtie_{\theta} E_2$

b. $\sigma_{\theta_1}(E_1 \bowtie_{\theta_2} E_2) \equiv E_1 \bowtie_{\theta_1 \wedge \theta_2} E_2$



Equivalence Rules (Cont.)

5. Theta-join operations (and natural joins) are commutative.



Equivalence Rules (Cont.)

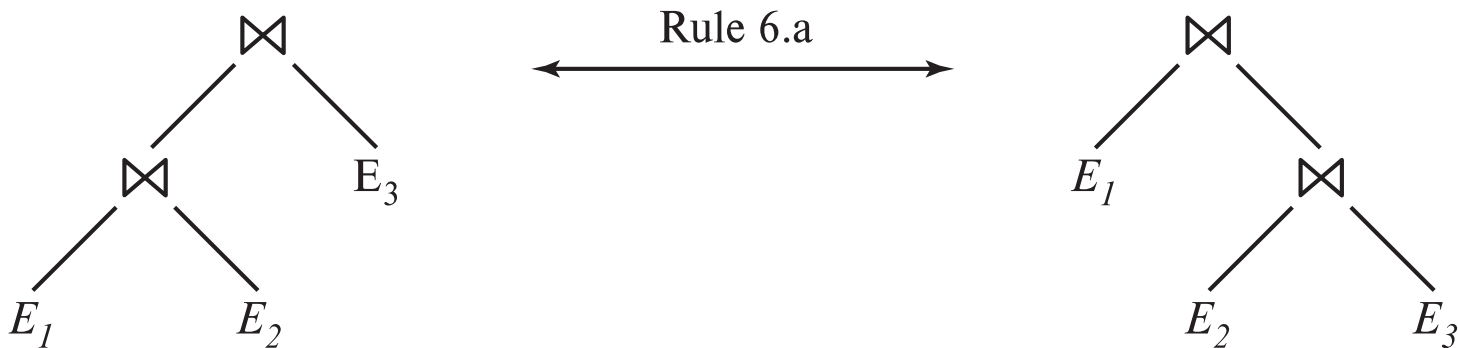
6. (a) Natural join operations are associative:

$$(E_1 \bowtie E_2) \bowtie E_3 \equiv E_1 \bowtie (E_2 \bowtie E_3)$$

(b) Theta joins are associative in the following manner:

$$(E_1 \bowtie_{\theta_1} E_2) \bowtie_{\theta_2 \wedge \theta_3} E_3 \equiv E_1 \bowtie_{\theta_1 \wedge \theta_3} (E_2 \bowtie_{\theta_2} E_3)$$

where θ_2 involves attributes from only E_2 and E_3 .

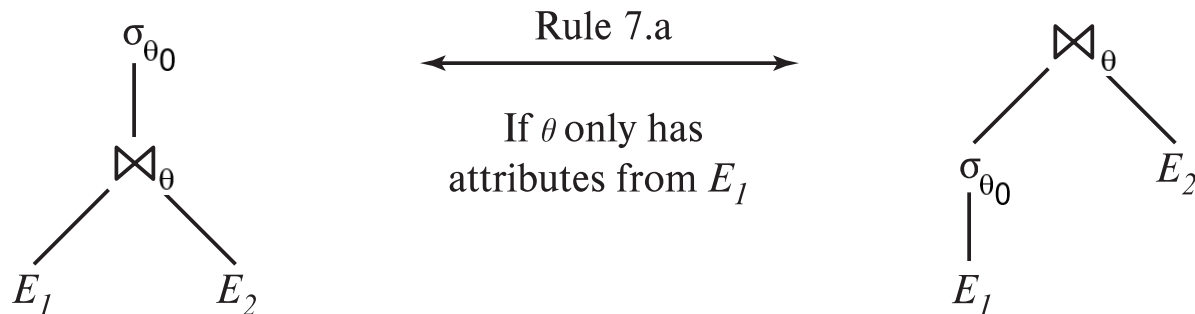


Equivalence Rules (Cont.)

7. The selection operation distributes over the theta join operation under the following two conditions:
- (a) When all the attributes in θ_0 involve only the attributes of one of the expressions (E_1) being joined.

$$\sigma_{\theta_0}(E_1 \bowtie_{\theta} E_2) \equiv (\sigma_{\theta_0}(E_1)) \bowtie_{\theta} E_2$$

similarly: $\sigma_{\theta_0 \wedge \theta}(E_1 \times E_2) \equiv \sigma_{\theta}((\sigma_{\theta_0}(E_1)) \times E_2)$



- (b) When θ_1 involves only the attributes of E_1 and θ_2 involves only the attributes of E_2 .

$$\sigma_{\theta_1 \wedge \theta_2}(E_1 \bowtie_{\theta} E_2) \equiv (\sigma_{\theta_1}(E_1)) \bowtie_{\theta} (\sigma_{\theta_2}(E_2))$$



Equivalence Rules (Cont.)

8. The projection operation distributes over the theta join operation as follows:

(a) if θ involves only attributes from $L_1 \cup L_2$:

$$\Pi_{L_1 \cup L_2}(E_1 \bowtie_{\theta} E_2) \equiv \Pi_{L_1}(E_1) \bowtie_{\theta} \Pi_{L_2}(E_2)$$

(b) In general, consider a join $E_1 \bowtie_{\theta} E_2$.

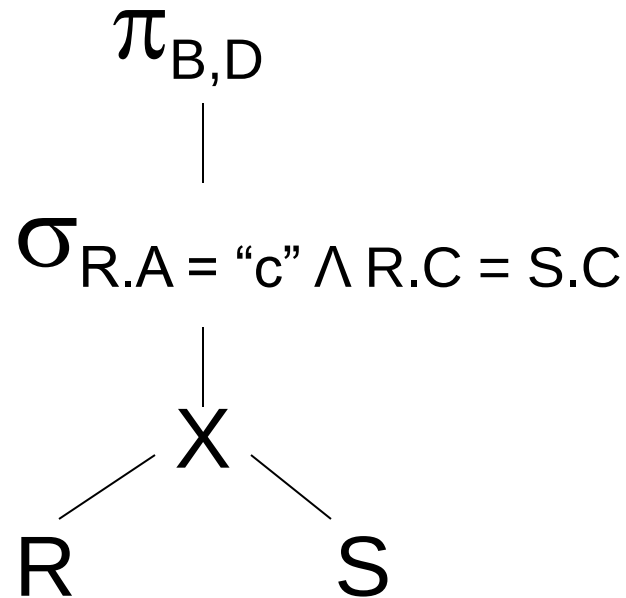
- Let L_1 and L_2 be sets of attributes from E_1 and E_2 , respectively.
- Let L_3 be attributes of E_1 that are involved in join condition θ , but are not in $L_1 \cup L_2$, and
- Let L_4 be attributes of E_2 that are involved in join condition θ , but are not in $L_1 \cup L_2$.

$$\Pi_{L_1 \cup L_2}(E_1 \bowtie_{\theta} E_2) \equiv \Pi_{L_1 \cup L_2}(\Pi_{L_1 \cup L_3}(E_1) \bowtie_{\theta} \Pi_{L_2 \cup L_4}(E_2))$$

Similar equivalences hold for outerjoin operations: $\bowtie$, $\ltimes$, and $\ltimes$

Query Transformation – Example 1

Select B,D
From R,S
Where R.A = "c" $\wedge$
R.C=S.C

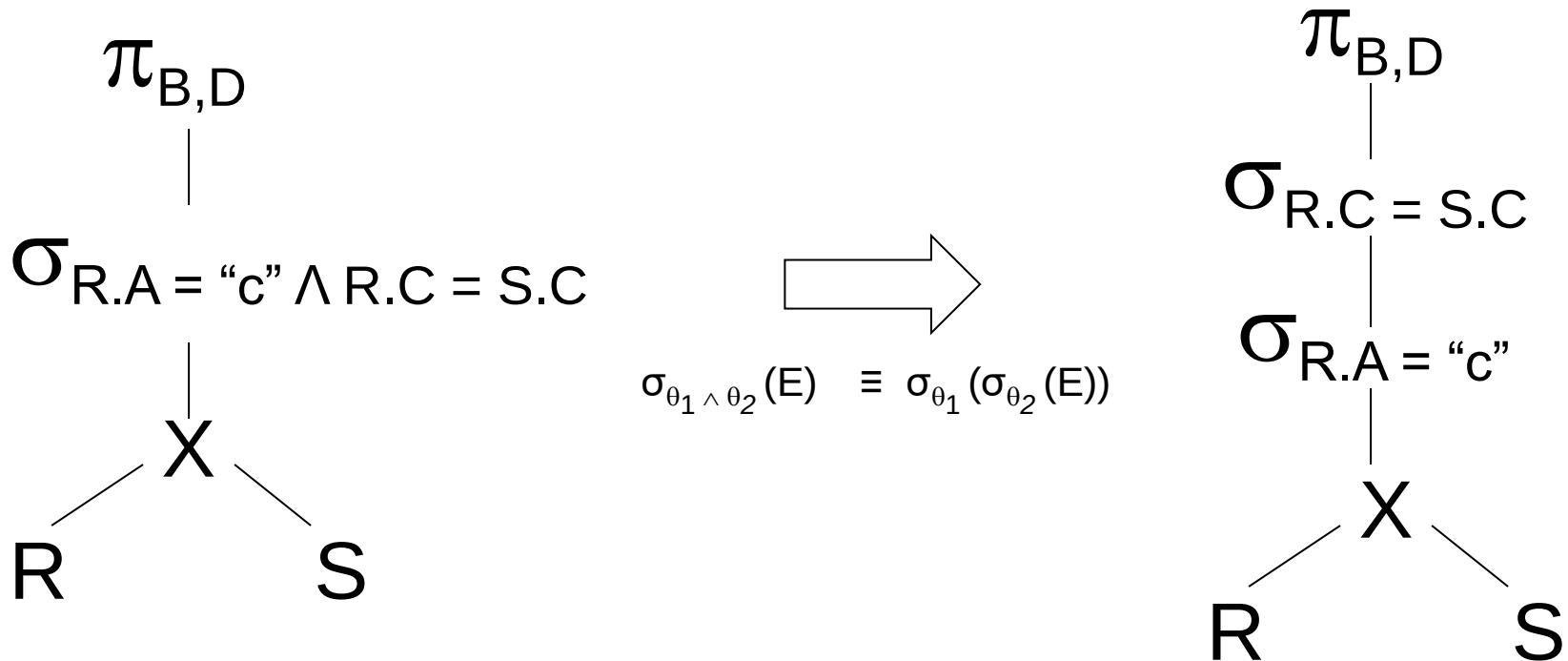


Initial Logical Plan

Relational Algebra: $\Pi_{B,D} [\sigma_{R.A=\text{"c"} \wedge R.C = S.C} (RXS)]$



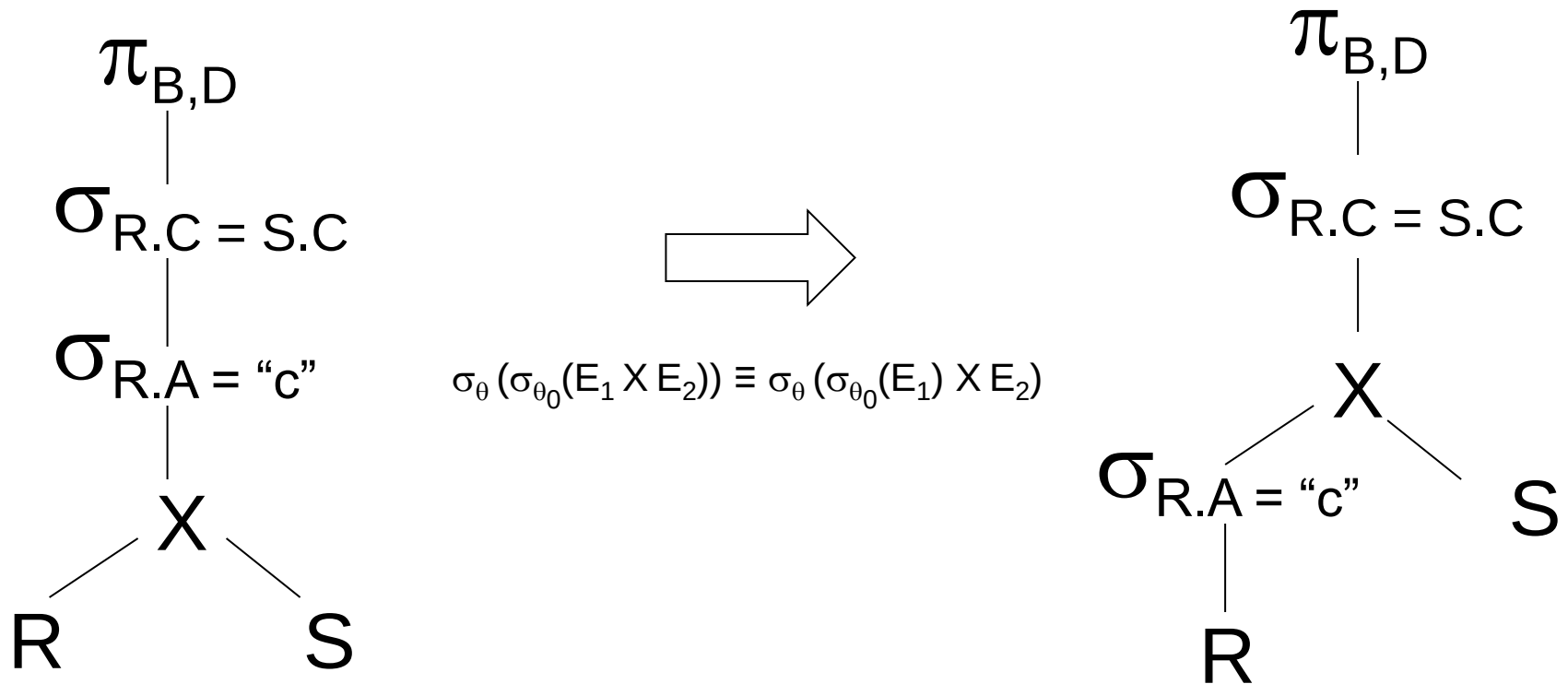
Apply Rewrite Rule (Step 1)



$$\Pi_{B,D} [\sigma_{R.C=S.C} [\sigma_{R.A=\text{"c"}}(R \times S)]]$$

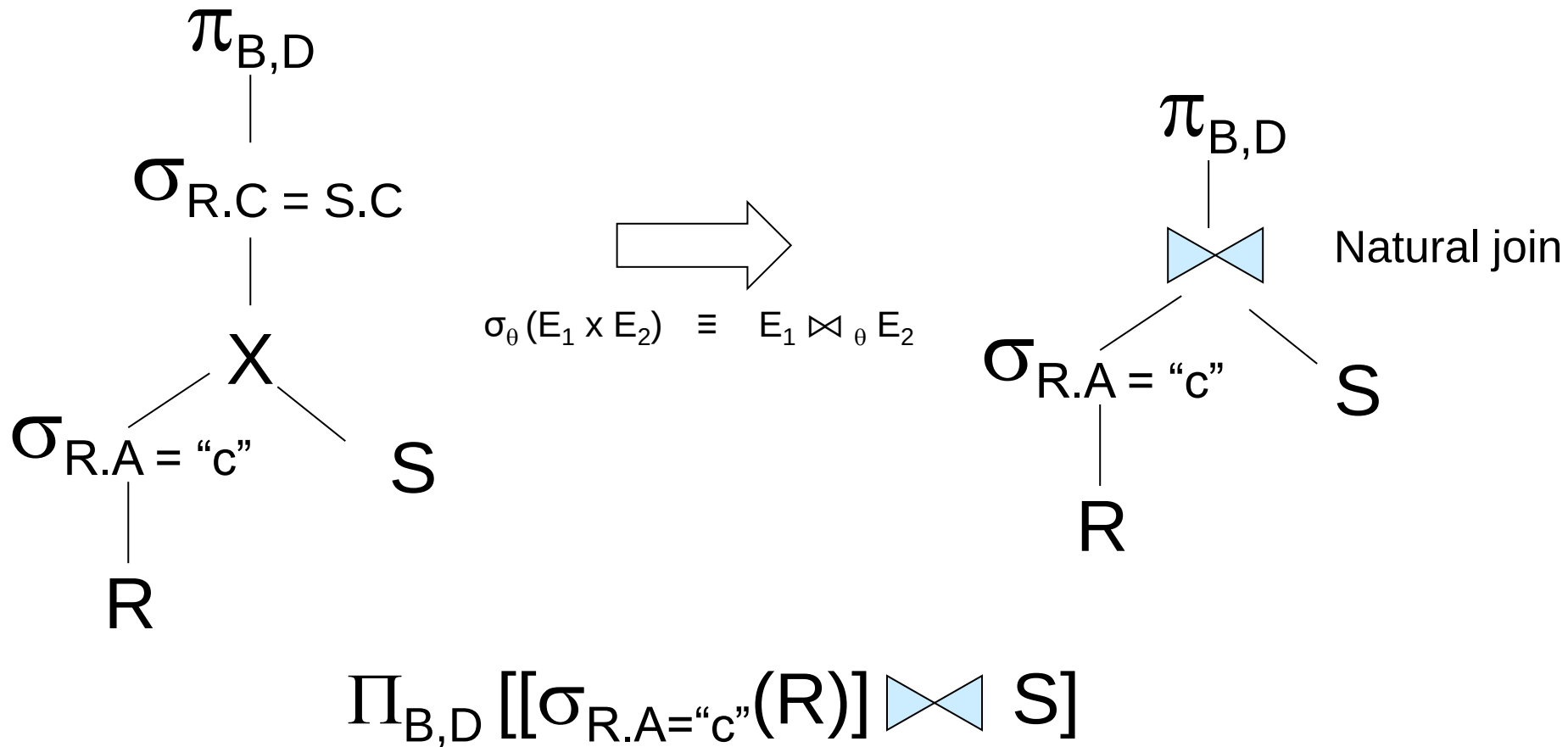


Apply Rewrite Rule (Step 2)



$$\Pi_{B,D} [\sigma_{R.C=S.C} [\sigma_{R.A='c'}(R)] \times S]$$

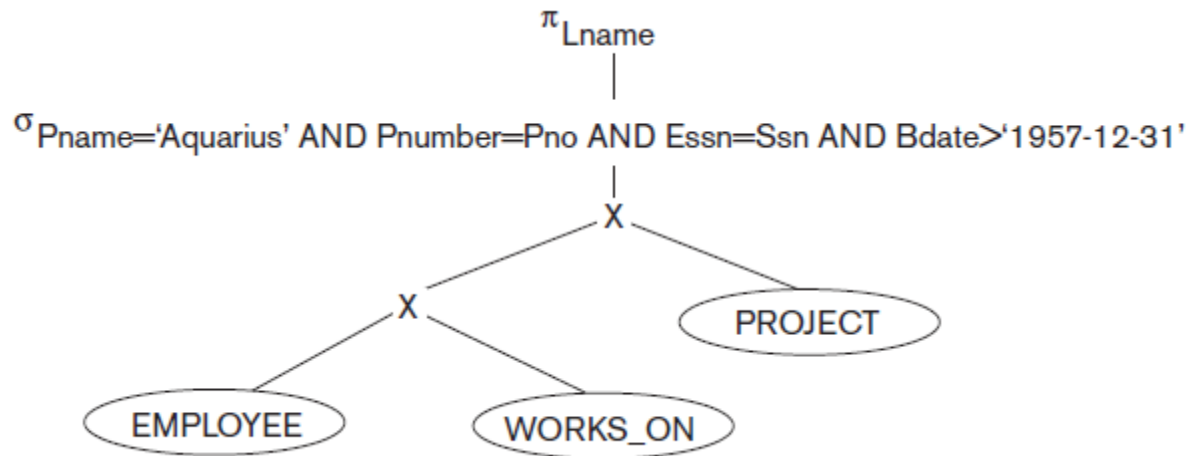
Apply Rewrite Rule (Step 3)



Query Transformation - Example 2

Q: SELECT E.Lname
 FROM EMPLOYEE E, WORKS_ON W, PROJECT P
 WHERE P.Pname='Aquarius' AND P.Pnumber=W.Pno AND E.Essn=W.Ssn
 AND E.Bdate > '1957-12-31';

(a)

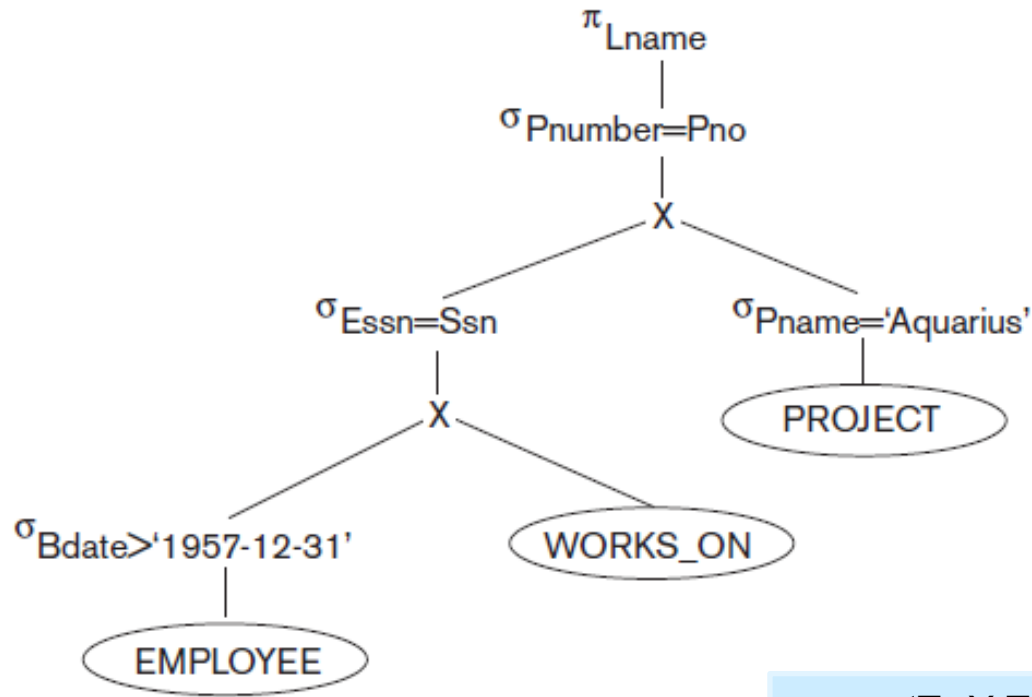


Steps in converting a query tree during heuristic optimization. (a) Initial (canonical) query tree for SQL query Q.



Query Transformation - Example 2 (cont'd.)

(b)



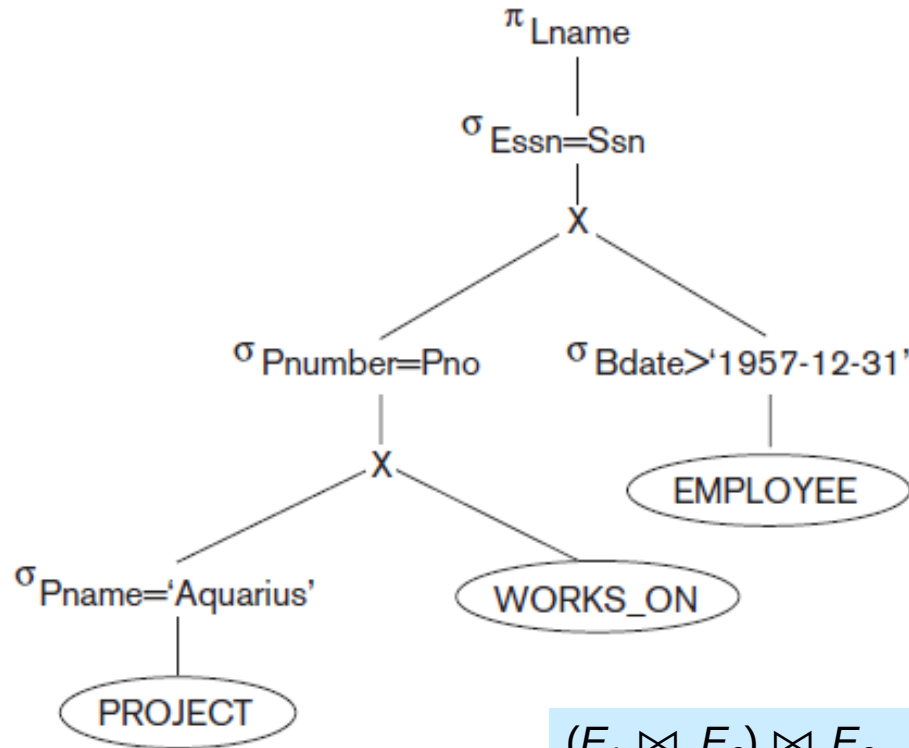
$$\sigma_{\theta_0 \wedge \theta} (E_1 \times E_2) \equiv \sigma_{\theta} ((\sigma_{\theta_0}(E_1)) \times E_2)$$

Steps in converting a query tree during heuristic optimization

(b) Moving SELECT operations down the query tree.

Query Transformation - Example 2 (cont'd.)

(c)



$$(E_1 \bowtie E_2) \bowtie E_3 \equiv E_1 \bowtie (E_2 \bowtie E_3)$$

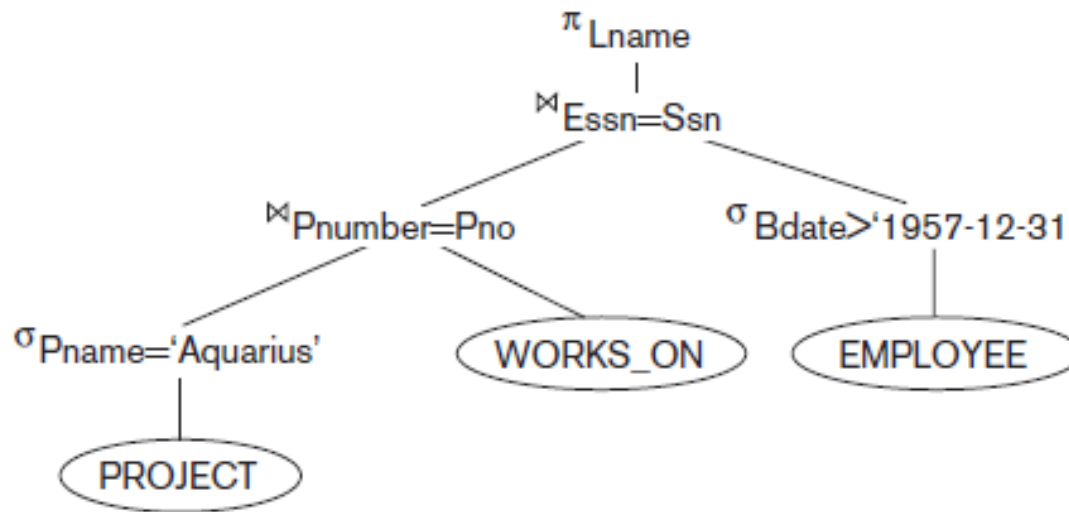
Steps in converting a query tree during heuristic optimization

(c) Applying the more restrictive SELECT operation first.



Query Transformation - Example 2 (cont'd.)

(d)



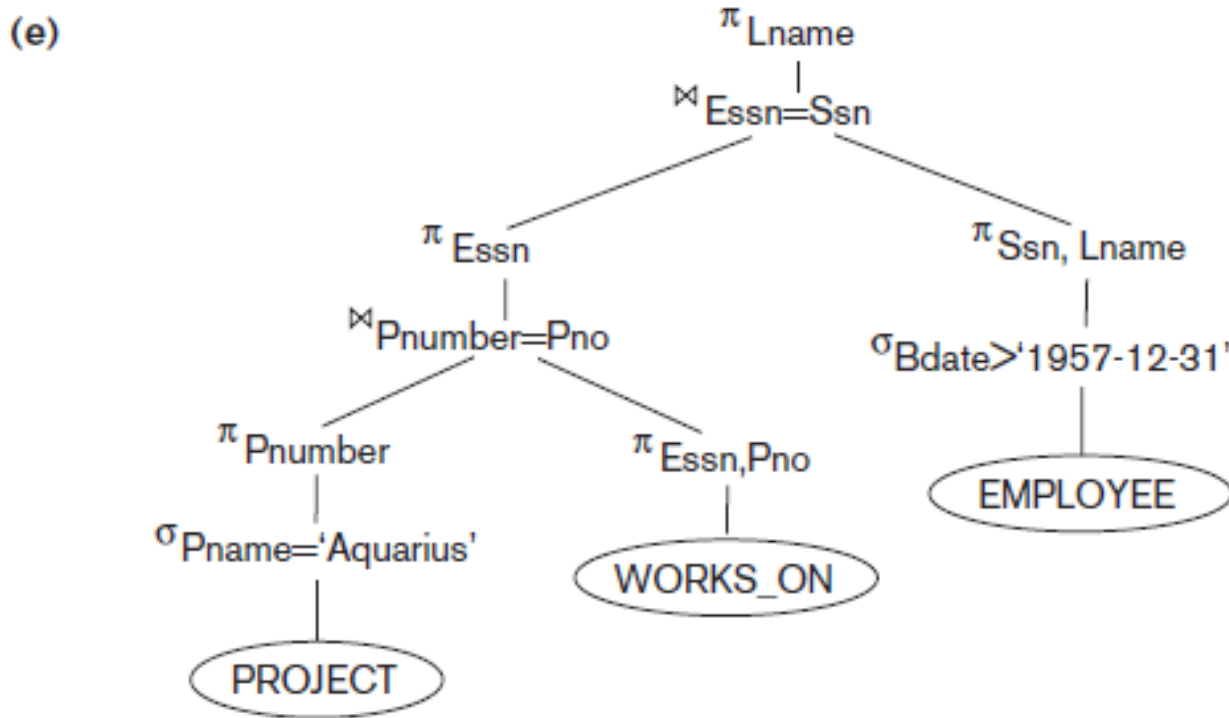
$$\sigma_{\theta}(E_1 \times E_2) \equiv E_1 \bowtie_{\theta} E_2$$

Steps in converting a query tree during heuristic optimization

(d) Replacing CARTESIAN PRODUCT and SELECT with JOIN operations.



Query Transformation - Example 2 (cont'd.)



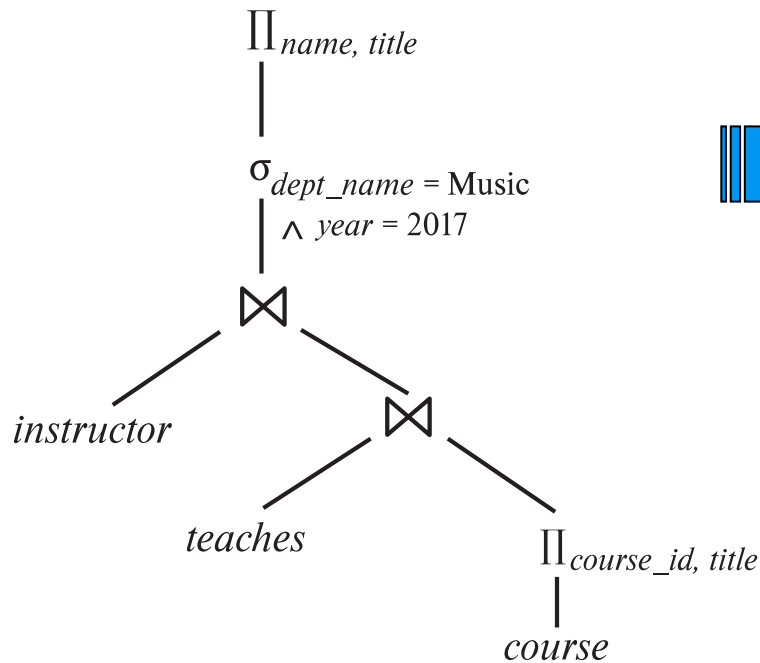
$$\Pi_{L_1 \cup L_2}(E_1 \bowtie_{\theta} E_2) \equiv \Pi_{L_1 \cup L_2}(\Pi_{L_1 \cup L_3}(E_1) \bowtie_{\theta} \Pi_{L_2 \cup L_4}(E_2))$$

Steps in converting a query tree during heuristic optimization

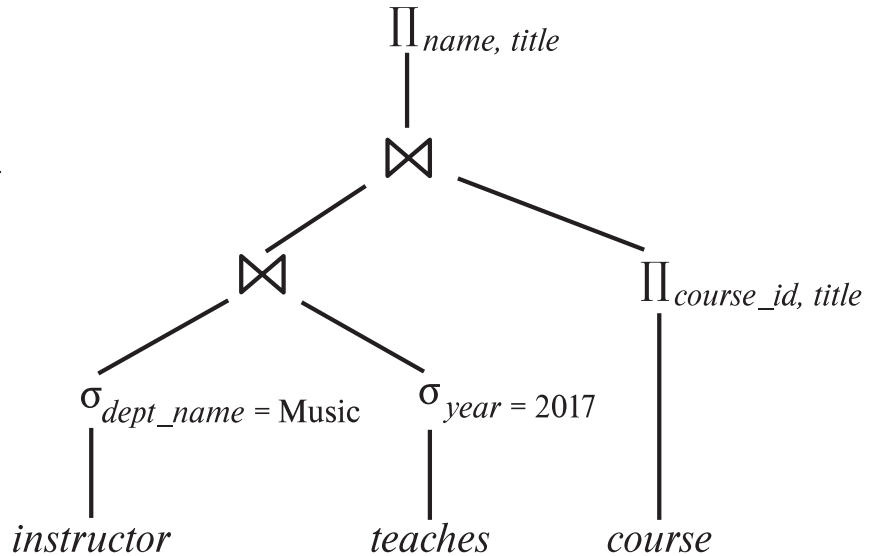
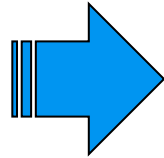
(e) Moving PROJECT operations down the query tree.



Query Transformation - Example 3



(a) Initial expression tree



(b) Tree after multiple transformations

